Overnet presenta:

Claude Code: sviluppo software con AI agentica

di Antonio Piemontese

Ottobre 2026







Indice dei contenuti del corso

01
Introduzione al corso ›

02
Cosa è la AI agentica: altro ›

03
Claude Code a 360° ›

04
Il Vibe Coding ›

05
Corso – slide operative ›

06
Varie ›



Introduzione al corso

Martedì scorso, uno sviluppatore Python di nome Marco ha passato **45 minuti** a discutere con ChatGPT di un bug nella sua app Flask...



Ha incollato il traceback. ChatGPT gli ha dato una correzione.

La correzione ha rotto qualcosa in un file che il modello non conosceva.



Ha incollato anche quell'errore —
Il modello si è scusato, ha riscritto la funzione da zero e ha ignorato gli ultimi tre messaggi.

Marco ha provato a spiegare la struttura del progetto. A metà strada il modello ha perso il contesto. Ha aperto un nuovo thread.
Ha incollato di nuovo il codice. Ha ottenuto una risposta diversa, in contraddizione con la prima.



Ha chiuso la scheda, ha aperto l'editor e ha corretto il bug da solo. Ci ha messo dodici minuti.

Marco non è un cattivo sviluppatore. Rilascia codice ogni settimana. Scrive in Python da sei anni.
Usa l'AI tutti i giorni.



E ogni giorno gli passa per la testa lo stesso pensiero: *deve esserci un modo migliore di fare tutto questo.*



Ha ragione. C'è.

Di recente abbiamo intervistato **278** sviluppatori Python. La maggior parte di loro somigliava molto a Marco:



«L'AI va bene per risolvere un piccolo compito, ma solo per quello.»



«Portare avanti una lunga sessione, poi sbattere contro i limiti del contesto e dover ricominciare da capo in un nuovo thread.»



«È facilissimo perdere di vista l'architettura di un'applicazione. C'è una frustrazione molto particolare quando qualcosa si rompe e non si ha idea di come risalire il filo del codice.»

Lo schema è sempre lo stesso.

L'AI basata su chat non vede il progetto. Non sa quali file esistono, non può eseguire i test e dimentica tutto quando la finestra di contesto si riempie. Così il lavoro di integrazione ricade su di noi: copiare codice da una finestra all'altra, rispiegare da capo il progetto, rivedere un output di cui non ci si fida fino in fondo.



65% degli sviluppatori del nostro sondaggio è fermo esattamente a questo punto. Usano l'AI per programmare, li aiuta nelle piccole cose, ma crolla davanti a qualsiasi cosa reale.



E sotto a tutto questo, una domanda che la maggior parte degli sviluppatori non pronuncia ad alta voce: *se non riesco a capire come lavorare con questi strumenti, che ne sarà della mia carriera?*

Esiste un modo diverso di lavorare.

Due note di traduzione: abbiamo lasciato invariati i tecnicismi d'uso corrente in italiano (traceback, thread, editor) e reso "context window" con "finestra di contesto", coerentemente con la terminologia usata nei corsi. Se serve, possiamo anche riunire tutte le sezioni tradotte finora in un unico documento impaginato.



Philipp Acsany

Real Python core team member. Uses Claude Code daily to build tools, automate workflows, and ship Python projects.

Invece di copiare codice da una finestra di chat, lavoriamo con un agente dentro il nostro progetto

Con Claude Code, l'AI gira nel nostro terminale. Legge i nostri file, li modifica direttamente, esegue i test, vede gli errori e li corregge. Gestisce git e lavora sull'intera codebase, non su un frammento alla volta.

Due scelte di traduzione da segnalare: "scaffold" è reso con "creare da zero la struttura / creare l'impalcatura" (è il termine tecnico per la generazione automatica dello scheletro iniziale di un progetto), e "skills" è reso con "Skill" maiuscolo perché "portable toolkit of reusable skills" si riferisce quasi certamente alla funzionalità Skill di Claude Code vista nella slide precedente, non a competenze generiche.

CHE COS'È L'AI AGENTICA?

Un sistema AI che persegue un obiettivo, sceglie le azioni da compiere e usa strumenti, adattando il percorso ai risultati con un grado di autonomia definito.

GLI ELEMENTI

MODELLO AI
Spesso un modello linguistico (LLM): interpreta e sceglie i passi.

CONTESTO
Istruzioni, dati e risultati del lavoro già svolto.

STRUMENTI
Consentono di operare su file, codice, web e applicazioni.

Il modello sceglie le azioni; il software le esegue.

IL CICLO DELL'AGENTE

1. PIANIFICA
Sceglie il prossimo passo.

2. AGISCE
Usa gli strumenti disponibili.

OBIETTIVO

3. VERIFICA
Controlla esiti e progressi.

4. ADATTA
Rivede il piano in base agli esiti.

Ripete se serve; conclude o si ferma ai limiti impostati.

UN ESEMPIO CONCRETO



«Analizza questi file e prepara un rapporto.»



Legge i file autorizzati.



Sceglie ed esegue le analisi.



Controlla dati e calcoli.



Prepara il rapporto.

Se mancano informazioni, può chiedere chiarimenti.

IL CONTROLLO UMANO

La persona definisce obiettivi, permessi e limiti, approva le azioni quando previsto e valuta il risultato.



IL PANORAMA DELL'AI AGENTICA

Ottobre 2026 - I Principali Strumenti e Framework per Agenti Autonomi

(aggiornato a ottobre 2026)

MULTI-AGENT FRAMEWORKS
(Coordinamento)

crewai
Framework orchestrazione agenti.

AG AutoGen
Sviluppo di flussi multi-agente di Microsoft

LangGraph
Agenti a stati complessi e ciclici.

AGENTIC CODING & TASK AGENTS
(I Tool più Importanti e Rivoluzionari)

FOCUS PRINCIPALE - OTTOBRE 2026



Claude Code (Anthropic)
L'agente CLI autonomo di Anthropic. Naviga, modifica, testa ed esegue comandi direttamente da terminale nel codebase



OpenAI Codex
L'ambiente agentico e di sviluppo di OpenAI. Risolve task complessi, corregge vulnerabilità e scrive software in autonomia

DATA & MEMORY AGENTS
(Integrazione RAG)

LlamaIndex
Integrazione di dati e RAG avanzato

LangChain
Libreria per catene di prompt e interazione con LLM.

ENTERPRISE SUITES

Semantic Kernel
Integrazione enterprise in .NET e Python.

PydanticAI
Validazione dati e sviluppo agenti basato su Pydantic.

AGENT OPS & EVALUATION
(Monitoraggio)

AgentOps
Monitoraggio, tracciamento e debugging di agenti.

CHE COSA È CLAUDE CODE

Una sintesi pratica per orientarsi rapidamente

In breve

Claude Code è un assistente di coding agentico: legge il contesto del progetto, conversa con l'utente e può eseguire azioni utili per analizzare, scrivere, modificare e verificare il codice.

1. COSA FA

- Legge file e struttura del progetto
- Propone o modifica codice
- Esegue comandi e controlli
- Aiuta nel debug e nel refactoring

2. COME SI USA

1. Apri Claude Code
2. Descrivi l'obiettivo
3. Claude legge, propone, agisce
4. Tu verifichi e iteri

È una collaborazione tra utente e AI.

PAROLE CHIAVE

- prompt
- contesto
- tool
- workflow
- iterazione
- verifica

3. DOVE LAVORA

- Nel terminale / CLI
- Con editor e IDE integrati
- Su file, repository e tool del progetto

4. PERCHÉ È UTILE

- Accelera attività ripetitive
- Riduce il lavoro manuale
- Aiuta a esplorare codice sconosciuto
- Supporta prototipi, fix e miglioramenti

5. COSA NON È

- Non sostituisce il giudizio umano
- Non garantisce che tutto sia corretto
- Non conosce automaticamente tutto il progetto senza leggerlo
- Non elimina la necessità di test e revisione

Idea centrale | Claude Code non è solo una chat: è un ambiente operativo per lavorare sul software in modo conversazionale, progressivo e assistito dall'AI.



Gian Segato
Ricercatore Anthropic

“Se puoi farlo col computer, puoi farlo con Claude Code".
Era chiaro che Claude Code sarebbe diventato una sorta di nuovo sistema operativo. Cioè l'idea di interagire col computer attraverso il linguaggio naturale. Il punto fondamentale è che tutti i lavori d'ufficio oggi hanno un qualche tipo di componente informatica. Lo facciamo dal computer e l’Informatica cos'è? Codice.

AI IN CLOUD vs AI AGENTICA

Differenze pratiche e quando usare Claude AI o Claude Code

Idea chiave
Claude AI e Claude Code non sono concorrenti diretti:
il primo è una superficie conversazionale general purpose,
il secondo è un ambiente agentico operativo per lavorare sul software.

1. DUE MODI DI USARE L'AI

AI in cloud · Claude AI

- Interazione soprattutto conversazionale
- Ottima per domande, testi, riassunti e brainstorming
- Lavora bene su prompt, file allegati e conoscenza generale
- Di solito non opera direttamente nel tuo progetto locale
- Più adatta a ragionare, spiegare, scrivere e analizzare

Punti forti
- semplicità
- rapidità
- versatilità
- accesso immediato

Dalla sola conversazione all'azione sul progetto

AI agentica · Claude Code

- Interazione conversazionale + azione operativa
- Legge contesto, file e struttura del progetto
- Può usare tool, terminale e workflow guidati
- Propone, modifica, verifica e itera sul codice
- Più adatta a sviluppo software e lavoro su repository locale o GitHub.com

Punti forti
- contesto del progetto
- automazione operativa
- iterazione
- debug e refactoring

2. CARATTERISTICHE A CONFRONTO

Claude AI	Voce	Claude Code
capire, spiegare, creare contenuti	**Obiettivo**	costruire, modificare e migliorare software
prompt e allegati	**Contesto**	prompt + file + struttura + tool
suggerisce	**Azione**	suggerisce e agisce
web / app cloud	**Ambiente**	terminale, editor, IDE, file locali e repository
analisi generale	**Tipo di lavoro**	workflow tecnico e iterativo
revisione delle risposte	**Controllo umano**	revisione di codice, azioni e risultati

3. QUANDO USARE CLAUDE AI

- **Per fare domande e capire un argomento**
- Per riassumere documenti o appunti
- Per scrivere, riscrivere o tradurre testi
- Per brainstorming e idee iniziali
- **Per valutare strategie prima di passare all'implementazione**

4. QUANDO USARE CLAUDE CODE

- **Per avviare un progetto software**
- Per esplorare codice esistente
- Per scaffolding, workflow e organizzazione
- **Per iterare, fare debug** e refactoring
- Per eseguire controlli e verifiche
- Per lavorare su repository locale o GitHub.com
- Per lavorare direttamente sui file locali
- Per evitare il **continuo** copia-incolla tra chatbot e IDE

5. PERCHÉ USARE CLAUDE CODE ANZICHÉ CLAUDE AI

1. Perché non si limita a rispondere: lavora sul progetto
2. Perché usa il contesto reale del codice, non solo il prompt
3. Perché supporta un ciclo completo: leggere → proporre → agire → verificare
4. Perché è più adatto a task tecnici multi-step
5. Perché riduce il lavoro manuale nello sviluppo software
6. Perché lavora direttamente sui file locali
7. Perché riduce il continuo copia-incolla tra chatbot e IDE

Nota: Claude Code non sostituisce il giudizio del docente o dello sviluppatore. Anche nell'AI agentica restano essenziali verifica, test e supervisione umana.

In una frase | Usa **Claude AI** quando ti serve soprattutto **pensare** con l'AI. Usa **Claude Code** quando ti serve far lavorare l'AI sul software, sui **file locali e sul repository**.

Claude Code: dove rende di più

Uno strumento agentico: legge il codebase, modifica file, esegue comandi e si integra con gli strumenti di sviluppo.
Non si limita a suggerire: agisce.



TASK SU CUI È COSTRUITO

Lavoro multi-file su codebase esistenti
Riceve un obiettivo in linguaggio naturale, pianifica, modifica più file e verifica il risultato.

Debugging a ciclo chiuso
Parte da errore o sintomo, trova la causa nel repo, corregge e lancia i test.

Lavoro tedioso e ripetitivo
Test mancanti, fix di lint, merge conflicts, aggiornamento dipendenze, release notes.

Workflow Git
Staging, commit message, branch, pull request; utile anche in CI per review e issue triage.

Esplorazione di codice sconosciuto
Spiega architettura e flussi di un repository reale.

Automazione da riga di comando
Componibile in pipeline Unix, script e CI.

DOVE RENDE MENO

Contesto fuori dal repo
Se non gli indichiamo esplicitamente dove guardare, può perdere informazioni importanti.

Conoscenza generale o brainstorming puro
Per domande secche o idee generiche, la chat è spesso più economica e adatta.

Requisiti ambigui
Senza piano e criteri di qualità espliciti, tende a proporre quick fix e accumulare debito tecnico.

Sessioni lunghe e sporche
Il degrado del contesto è il rischio principale; aiutano /clear frequente e un CLAUDE.md asciutto.

REGOLA PRATICA

Se il task ha un criterio di verifica eseguibile — test che passano, **lint pulito, script che gira, diff che si applica** — **Claude Code è nel suo territorio migliore**, perché può iterare da solo fino al **verde.**



Se il criterio è soggettivo o esterno al progetto, spesso conviene restare in chat.

Riferimento ufficiale: code.claude.com/docs/en/overview

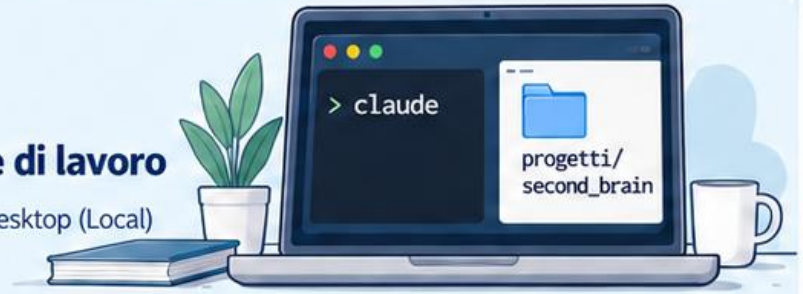
Claude Code: locale e cloud

Due ambienti complementari. La scelta dipende da dove devono lavorare codice e strumenti.

LOCALE

Controllo sull'ambiente di lavoro

Terminale · VS Code / JetBrains · Desktop (Local)



Esecuzione	Sul proprio PC, nella cartella del progetto.
Risorse	File, conda, notebook e GPU locali, secondo i permessi.
Configurazioni	Utente, progetto, locale e policy aziendali.
Integrazioni	Hook, MCP e plugin configurati sul PC.
Requisiti	Installazione locale. Installer nativo senza Node.js.
Ideale per	Sviluppo iterativo, debug, test e integrazione con l'IDE.

CLOUD

Delega su un ambiente remoto

Claude Code sul web · **claude.ai/code**



Esecuzione	VM isolata di Anthropic con clone del repository.
Risorse	File e tool della VM; il PC resta separato.
Configurazioni	Settings del repo committati, ambiente cloud e policy server.
Integrazioni	Hook e MCP del repo; plugin non installati automaticamente.
Requisiti	Percorso web: browser, piano abilitato e repo GitHub collegato.
Ideale per	Task delimitati e paralleli, anche a PC spento. Risultati da rivedere e possibile PR.

IL REPOSITORY PORTA LE REGOLE CONDIVISE

Commit e push su GitHub rendono disponibili istruzioni e impostazioni condivise alle sessioni cloud che clonano quel branch.

`CLAUDE.md`

`.claude/settings.json`

Restano sul PC:
`~/.claude/settings.json`
e `.claude/settings.local.json`

PER IL CORSO

Locale come base di lavoro · Cloud per attività delegabili

Locale indica dove girano strumenti e codice; il modello Claude è remoto.

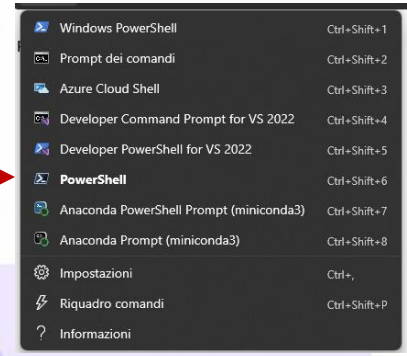
CLI e browser sono interfacce: il terminale può avviare il cloud; Remote Control può controllare una sessione locale.

Cloud: scenario Anthropic con un repository.

Fonti: documentazione ufficiale Claude Code.

Claude Code su Windows

La shell esegue i comandi. Windows Terminal organizza le sessioni.



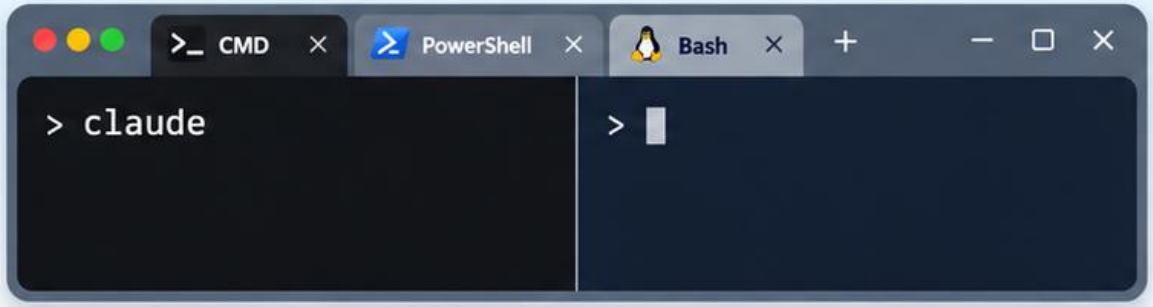
Non confondere con il vecchio Windows PowerShell

L'APP CHE OSPITA LE SHELL

Windows Terminal

App Terminale di Windows

Scaricabile dal Microsoft Store



Più schede
Più sessioni in una finestra

Riquadri affiancati
Sessioni visibili insieme

Profili per le shell
Un menu per ambienti diversi

Personalizzazione
Font, colori e scorciatoie

Profili per shell installate e configurate sul PC.

DA DOVE AVVIARE CLAUDE CODE

CMD
Prompt dei comandi
La shell classica di Windows.

PowerShell
Comandi e automazione
Shell con scripting avanzato.

Bash
Git Bash oppure Linux / WSL
Git Bash lavora su Windows.
Con WSL, installare Claude Code nell'ambiente Linux.

Anaconda Prompt
CMD con conda già attivo
Può avviare Claude Code.
Opzionale: utile per progetti Python.

PER INIZIARE

1 Aprire una shell → 2 Entrare nella cartella del progetto → 3 `claude` Avvia Claude Code

Requisito: Claude Code installato nell'ambiente scelto e comando `claude` disponibile nel PATH.

Fonti: Claude Code Docs · Microsoft Learn · Anaconda Docs

IDEE
DOMANDE
PROGETTI
RISULTATI
TUTTO INSIEME

Novità del 16 settembre

Claude unifica Chat e Cowork in un'unica esperienza

Un'unica conversazione. Più possibilità.

Anthropic ha annunciato che non è più necessario scegliere prima dove iniziare: la stessa conversazione può gestire sia domande rapide sia lavori più complessi.

1 Prima

L'utente doveva scegliere tra:

- **Chat** → per domande rapide
- **Cowork** → per attività più articolate

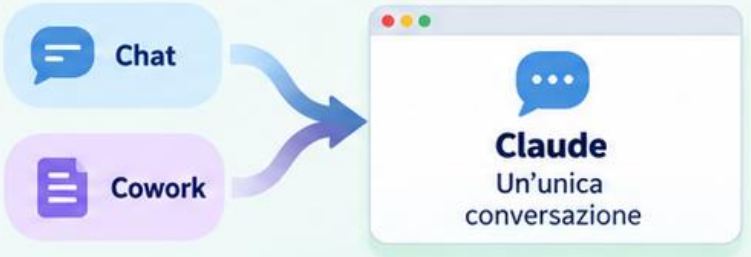
Chat
Domande rapide

oppure

Cowork
Attività più articolate

2 Ora

Chat e Cowork diventano un'unica esperienza.
Non c'è più una modalità da scegliere prima di iniziare.



3 Come funziona



Si chiede ciò che serve in un'unica conversazione.

Claude decide quali capacità usare per svolgere il compito.

Può rispondere a una domanda oppure portare avanti un lavoro più ampio.

4 Cosa può fare

- **Ricerca e sintesi**
- **Report e fogli di calcolo**
- **Presentazioni**
- **Lavori che continuano anche nel cloud**

Più attività possono essere gestite nella stessa conversazione.

5 Nuovi output

Arrivano anche:

- **Claude Docs**
- **Claude Slides**
- **Claude Design**

nella conversazione.

I risultati possono essere modificati, condivisi ed esportati (inclusi PowerPoint e PDF).

6 Importante

- Il rilascio è graduale
- Prima per i piani Pro e Max
- Su web, desktop e mobile
- Team e Free seguiranno dopo
- Chat, task, progetti e impostazioni restano disponibili

Per impostazione predefinita, Claude chiede conferma prima di eseguire azioni: il controllo resta all'utente.

In sintesi: dal 16 settembre Claude non separa più Chat e Cowork.
Una sola conversazione può rispondere, creare documenti e portare avanti attività complesse.

STESSE IDEE.
MAGGIORI POSSIBILITÀ.

PROGRAMMA DEL CORSO

Claude Code in 2 giornate

Panoramica sintetica del percorso: giovedì e venerdì

Obiettivo

Impostare le basi pratiche per lavorare con Claude Code, creare un primo progetto Python e imparare a raffinarlo progressivamente.

GIOVEDÌ · PRIMA GIORNATA

Dalle basi al primo progetto

1. Introduzione a Claude Code
2. Requisiti di installazione dei vari strumenti
3. Alcuni comandi iniziali particolari
4. Creazione del primo progetto Python
5. Scaffolding: infrastruttura del progetto
6. Costruzione di un workflow iniziale

Focus: avvio, setup e struttura

Setup

↓

Primo progetto

↓

Raffinamento

VENERDÌ · SECONDA GIORNATA

Raffinamento e concetti avanzati

1. Raffinamento del progetto creato il primo giorno
2. Iterazione sul progetto
3. Debug
4. Skill e skill agentiche
5. Buzzwords: alcuni termini di moda
6. Agenti e sub-agenti
7. Server MCP
8. Mappa finale dei termini

Focus: miglioramento, agenti e terminologia

Nota finale

Non ci sarà il tempo per vedere e implementare tutto. Alla fine della seconda giornata forniremo comunque una mappa dei termini per orientarsi tra i concetti principali.

COME L'AI HA CONTRIBUITO A QUESTE SLIDE

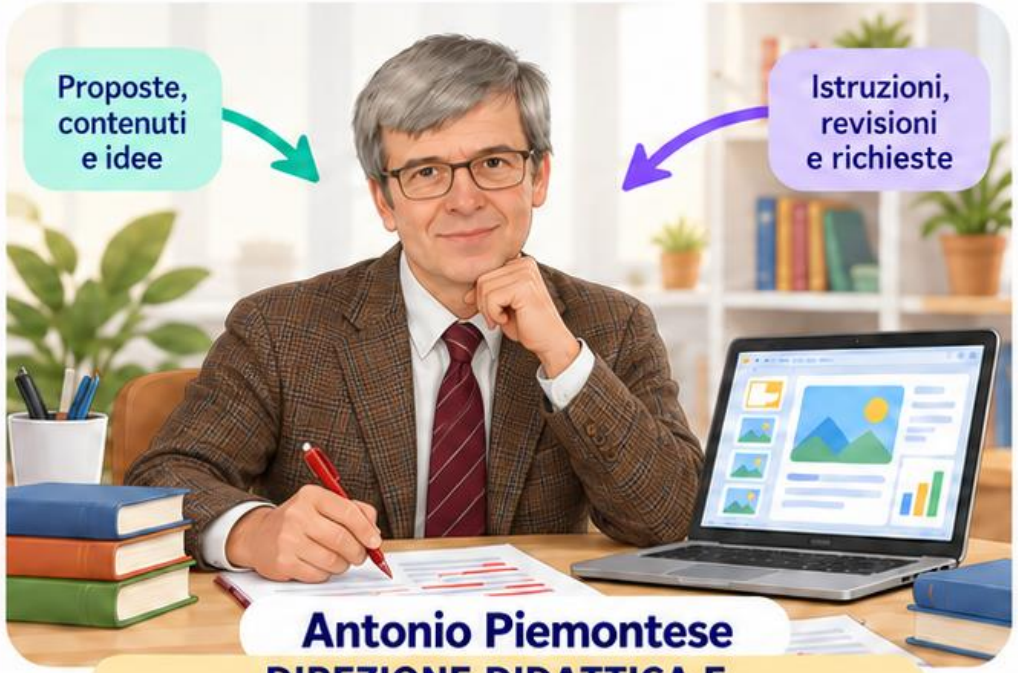
ChatGPT e Claude AI hanno affiancato il docente nella preparazione dei materiali didattici.

IL SUPPORTO DELL'AI

ChatGPT | Claude AI

- Stesura e sintesi dei testi
- Riformulazioni e traduzioni
- Elaborazione di esempi e schemi
- Preparazione dei contenuti visivi

SOTTO LA DIREZIONE DEL DOCENTE



Antonio Piemontese
DIREZIONE DIDATTICA E RESPONSABILITÀ DEI CONTENUTI

LE SCELTE DEL DOCENTE

- Ha definito obiettivi e impostazione.
- Ha guidato il lavoro con istruzioni.
- Ha richiesto modifiche e adattamenti.
- Ha selezionato i contenuti delle slide.

DICHIARAZIONE DI TRASPARENZA

Queste slide sono state preparate con il supporto di **ChatGPT e Claude AI, sotto la direzione del docente** Antonio Piemontese, che mantiene la responsabilità dei contenuti.

RIFERIMENTO ALL'AI ACT
Regolamento (UE) 2024/1689 — art. 50

Gli obblighi di trasparenza riguardano casi specifici. Per i testi pubblicati su temi di interesse pubblico è prevista un'esenzione in presenza di revisione umana o controllo editoriale, insieme alla responsabilità editoriale della pubblicazione.

Fonte normativa: Commissione europea — AI Act Service Desk, art. 50

Gli strumenti AI del corso

Esecuzione sul progetto e assistenza durante le attività

Claude Code

circa **70%**

ESECUZIONE SUL PROGETTO LOCALE

Claude Code viene eseguito sul PC.
Lavora sui file ed esegue le attività del progetto.

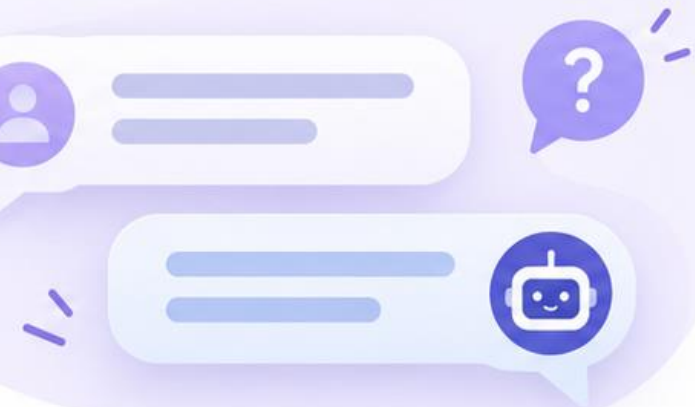


Copilot Chat

circa **30%**

ASSISTENTE AI

Offre spiegazioni e chiarimenti.
Aiuta a ragionare durante le attività.



Ripartizione indicativa dell'utilizzo durante il corso

CLAUDE CODE: PIANI DI ACCESSO

Sintesi del Markdown · Ottobre 2026 · Prezzi web in USD

PIANI INDIVIDUALI

Free
0 $

Claude Code: **NO**

Uso base di Claude.
La finestra d'uso si azzera ogni 5 ore.

Pro
20 $/mese

fatturazione mensile
oppure 200 $/anno anticipati
≈ 17 $/mese con annuale

Claude Code: **SÌ**

Almeno **5× Free** per sessione
+ limiti settimanali
Punto d'ingresso individuale

Max 5x
100 $/mese

solo fatturazione mensile

Claude Code: **SÌ**

5× Pro per sessione
100 $/mese di crediti API inclusi
Uso intensivo dalla CLI

Max 20x
200 $/mese

solo fatturazione mensile

Claude Code: **SÌ**

20× Pro per sessione
200 $/mese di crediti API inclusi
Uso intensivo dalla CLI

ORGANIZZAZIONI E ALTERNATIVA API

Team
2–150 posti

Claude Code: **SÌ**

Standard e Premium mescolabili

PREZZI PER POSTO AL MESE

Standard:	**20 $** con annuale · **25 $** con mensile
Premium:	**100 $** con annuale · **125 $** con mensile

Standard > Pro · Premium = 5× Standard
Fino a 500 $/mese di crediti API condivisi
SSO e amministrazione centralizzata

Enterprise
Claude Code: **SÌ**

20 $/posto/mese
+ consumo a tariffe API

Fatturazione annuale · minimo 20 posti
Tetti di spesa per utente e organizzazione
Governance avanzata: RBAC, SCIM e audit log

Console/API
Claude Code: **SÌ**

Pagamento per token

Nessun canone mensile

CLI · VS Code · JetBrains
Alternativa all'abbonamento

I prezzi sono quelli riportati nell'allegato e sono soggetti a modifica.
Sugli store mobili possono variare.

Fonte: "Piani di abbonamento Claude (ottobre 2026)" · @Antonio

Piani di abbonamento Claude (ottobre 2026)

19 ott 2026 ·

I prezzi sono pubblicati in dollari USA (la pagina ufficiale non riporta prezzi in euro), valgono per la sottoscrizione via web — sugli store mobili possono variare — e sono dichiarati soggetti a modifica a discrezione di Anthropic.

Tabella riassuntiva

Piano	Prezzo	Limiti d'uso	Claude Code
Free	0 $	Uso base, finestra che si azzera ogni 5 ore	No
Pro	17 $/mese con fatturazione annuale (200 $ anticipati) · 20 $/mese mensile	Almeno 5× il Free per sessione, più limiti settimanali	Sì
Max 5x	100 $/mese (solo mensile)	5× il Pro per sessione	Sì
Max 20x	200 $/mese (solo mensile)	20× il Pro per sessione	Sì
Team	Posto Standard: 20 $/posto/mese annuale (25 $ mensile) · Posto Premium: 100 $ annuale (125 $ mensile)	Standard: superiore al Pro · Premium: 5× lo Standard	Sì
Enterprise	20 $/posto/mese (annuale, minimo 20 posti) + consumo fatturato a tariffe API	Scala col consumo; gli admin impostano tetti di spesa per utente e organizzazione	Sì

Team è pensato per gruppi da 2 a 150 posti, con posti Standard e Premium mescolabili.

Piani individuali: offerta tecnica incrementale

Free — Modelli Sonnet e Haiku; fino a 5 Projects; memoria, connettori, ricerca web, creazione file ed esecuzione codice, Artifacts (Design, Slides, Docs), voice mode, skills; contesto fino a 1M di token (varia per modello, come su tutti i piani). Niente Claude Code né Research.

Pro (aggiunge al Free) — Opus incluso; Fable accessibile tramite *crediti d'uso* (consumo extra fatturato a tariffe API quando si esauriscono i limiti); **Claude Code** su tutte le superfici; Research; pianificazione e handoff dei task; Claude in Chrome, Claude per Microsoft 365/Outlook, Claude Science, Dashboards (beta). Nessun controllo amministrativo; training sui dati disattivabile (opt-out).

Max 5x / 20x (aggiunge al Pro) — Limiti 5× o 20×; **Fable incluso fino al 50% dei limiti settimanali**; crediti API inclusi: 100 $/mese (5x) o 200 $/mese (20x); accesso prioritario nelle ore di punta e accesso anticipato alle funzioni nuove. È il piano tipico per chi usa Claude Code in modo intensivo dalla CLI.

Piani per organizzazioni: Team ed Enterprise

Team (2–150 posti, Standard e Premium mescolabili) — Tutto quanto sopra più: condivisione di Projects, enterprise search nei connettori, Motion (beta), fino a 500 $/mese di crediti API *condivisi* dal team; Fable al 50% dei limiti settimanali sui posti Premium. Lato amministrazione: SSO, verifica del dominio, fatturazione centralizzata, controlli admin sui connettori, analytics d'uso, skills a livello di organizzazione; **nessun training sui dati per impostazione predefinita**. Non include: SCIM, audit log, Compliance API, retention personalizzata, offerta HIPAA. Pagamento con carta, mensile o annuale (niente bonifico/fattura).

Enterprise — Modello nuovo: quota posto bassa (20 $) ma **consumo fatturato a tariffe API**, con tetti di spesa impostabili dagli admin. Tutti i modelli inclusa Fable; tutto il Team più Claude Security (beta) e il pacchetto completo di governance: SSO, permessi granulari (RBAC), SCIM, audit log, Compliance API, retention personalizzata, controlli di rete e IP allowlisting, offerta HIPAA-ready, inference hooks. Fatturazione annuale (carta o ACH in self-serve; con il sales anche fattura con termini di pagamento).

Note pratiche per Claude Code

- Per usare Claude Code individualmente il **Pro** è il punto d'ingresso; **Max** è il piano tipico per l'uso intensivo dalla CLI.
- In alternativa all'abbonamento, Claude Code (CLI, VS Code, JetBrains) funziona anche con un account **Console/API a consumo**: si paga per token, senza canone mensile.

Fonti

Verificate il 19 ottobre 2026:

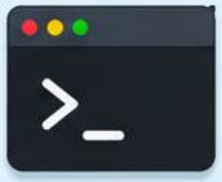
- Pricing — claude.com
- Che cos'è il piano Max — Supporto Claude
- Che cos'è il piano Pro — Supporto Claude

CLAUDE CODE: 6 MODALITÀ DI ACCESSO

Le “superfici” descritte nel Markdown • Ottobre 2026

STESSO MOTORE | Configurazione del repository: CLAUDE.md • impostazioni • server MCP

raccomandato!



1. Terminale (CLI)

Avvio nel progetto: `claude`
Funzioni complete, script, pipe e SSH.
Print mode: `-p`

Richiede pratica; diff testuali.
Windows: Git for Windows per Bash.



2. VS Code / Cursor

Estensione con diff inline e revisione dei piani.
File e codice nel contesto dell’editor.

Legata al progetto aperto; poca automazione.



3. JetBrains

IntelliJ IDEA • PyCharm • WebStorm

Diff interattivi e selezione come contesto.

Richiede la CLI installata separatamente.



4. App desktop

macOS • Windows • Ubuntu/Debian beta

File locali, sessioni parallele e pianificate.
Anche sessioni cloud.

Abbonamento a pagamento.
CLI separata non necessaria.



5. Web

`claude.ai/code`

Zero installazione; task cloud in parallelo.
Prosegue anche a computer spento.

Repository collegati: niente accesso diretto a file e strumenti locali.



6. App mobile

iOS • Android

- Sessioni web → cloud
- Dispatch → task al desktop
- Remote Control → continua una sessione locale
- Avvio e supervisione ovunque.

Schermo piccolo per i diff.

ACCOUNT E PROVIDER | In genere: abbonamento Claude o account Console.
CLI, VS Code e JetBrains supportano anche provider terzi.

INTEGRAZIONI: PIPELINE E STRUMENTI

- **GitHub Actions / GitLab CI-CD** — Review e triage delle issue
- **GitHub Code Review** — Revisione automatica delle pull request
- **Slack** — @Claude: da bug report a pull request
- **Chrome** — Debug di web app nel browser
- **Channels** — Eventi da Telegram, Discord, iMessage e webhook
- **Agent SDK** — Agenti personalizzati

NELLA PRATICA | **CLI + editor:** lavoro quotidiano e didattica • **Desktop:** via grafica • **Web e mobile:** task lunghi e lavoro da fuori | **Fonte:** “Modalità di accesso a Claude Code” • @Antonio • 19 ottobre 2026

Cosa è l’AI agentica: altro

Le skill degli agenti, spiegate chiaramente

contiene errori, è riferita a OpenAI

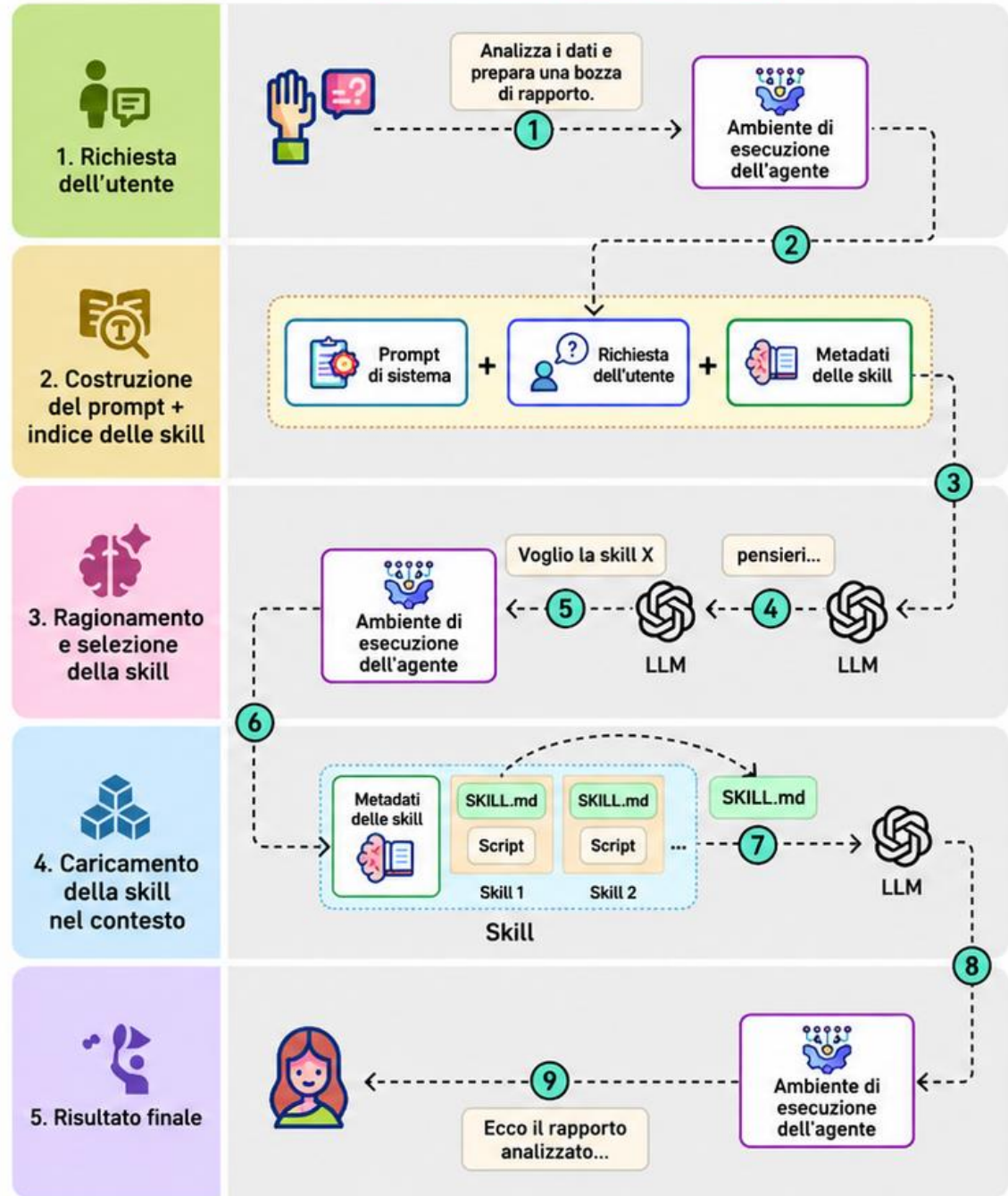
Perché servono le skill degli agenti? I prompt lunghi peggiorano le prestazioni degli agenti. Al posto di un unico prompt enorme, gli agenti mantengono un piccolo catalogo di skill: procedure riutilizzabili con istruzioni chiare, caricate solo quando necessario.

Ecco come funziona il flusso di lavoro delle skill degli agenti:

1. Richiesta dell'utente: un utente invia una richiesta come «Analizza i dati e prepara una bozza di rapporto».
2. Costruzione del prompt + indice delle skill: l'ambiente di esecuzione dell'agente combina la richiesta con i metadati delle skill: un elenco essenziale delle skill disponibili e delle loro brevi descrizioni.
3. Ragionamento e selezione della skill: l'LLM elabora il prompt, ragiona e decide: «Voglio la skill X».
4. Caricamento della skill nel contesto: l'ambiente di esecuzione dell'agente riceve dall'LLM la richiesta di una specifica skill. Quindi carica il file SKILL.md e lo aggiunge al contesto attivo dell'LLM.
5. Risultato finale: l'LLM segue SKILL.md, esegue gli script e genera il rapporto finale.

Le skill degli agenti vengono caricate dinamicamente solo quando necessario, mantenendo ridotto il contesto e coerente il comportamento dell'LLM.

La parola a voi: quali skill trovereste più utili negli agenti?



AI AGENTICA CON CLAUDE CODE · MODULO SKILLS

Agent Skills → Claude Code

Lo stesso flusso in cinque passi: a sinistra il pattern generico delle Agent Skills, a destra il meccanismo con cui lo re
Code.

Mapping della slide precedente su Claude Code (fatta da Claude AI)

ANATOMIA DI SKILL.MD

```
---
name: riassumi-modifiche
description: quando usarla
---
# istruzioni per Claude…
```

PATTERN GENERICO · LE AGENT SKILLS COME CONCETTO		IN CLAUDE CODE · IL MECCANISMO REALE
Richiesta dell'utente Una richiesta arriva all'agente. Per esempio: «Analizza i dati e prepara il report».	01 →	**Il prompt entra nella sessione** La richiesta viene scritta nella sessione di Claude Code (terminale, VS Code o app desktop). Nel contesto ci sono già il system prompt e le istruzioni di progetto di CLAUDE.md. `claude` `CLAUDE.md`
Prompt + indice delle skill Il runtime — l'infrastruttura che orchestra l'agente — unisce la richiesta a un indice leggero delle skill: solo nome e breve descrizione di ciascuna.	02 →	**In contesto va solo il «listing»** All'avvio della sessione entrano in contesto soltanto name e description di ogni skill, letti dal frontmatter: l'intestazione YAML in cima a SKILL.md. Il corpo del file resta fuori. `name:` `description:` `frontmatter`
Ragionamento e selezione Il modello valuta la richiesta rispetto all'indice e decide quale skill gli serve: «voglio la Skill X».	03 →	**Claude sceglie — o si invoca a mano** Claude confronta la richiesta con le description e invoca la skill pertinente; in alternativa la si richiama con /nome-skill. Due campi del frontmatter stabiliscono chi può invocarla. `/nome-skill` `disable-model-invocation` `user-invocable`
Caricamento della skill Il runtime riceve la scelta e carica il file di istruzioni della skill — SKILL.md — nel contesto attivo del modello.	04 →	**SKILL.md entra in conversazione** Il corpo di SKILL.md si carica solo all'invocazione e resta in contesto anche nei turni successivi. I file di supporto — riferimenti, esempi, script — vengono letti solo se servono davvero. `SKILL.md` `reference.md` `scripts/`
Esecuzione e output Il modello segue le istruzioni della skill, esegue gli script previsti e produce il risultato finale.	05 →	**Claude esegue con i suoi tool** I passi vengono svolti con i tool della sessione (Bash, Read, Edit…); il campo allowed-tools può pre-approvare i permessi per il turno dell'invocazione. Il risultato torna all'utente. `allowed-tools` `Bash` `Edit`

PROGRESSIVE DISCLOSURE
«rivelazione progressiva»: ogni livello entra in contesto solo quando serve davvero.

LIVELLO 1 · SEMPRE IN CONTESTO
Metadati: name + description

LIVELLO 2 · ALL'INVOCAZIONE
Corpo di SKILL.md

LIVELLO 3 · SOLO SE SERVONO
File di supporto: esempi, script

RISULTATO
Contesto corto → prestazioni migliori e comportamento coerente.

Infografica originale per il corso · terminologia dalla documentazione ufficiale di Claude Code

code.claude.com/docs/en/skills

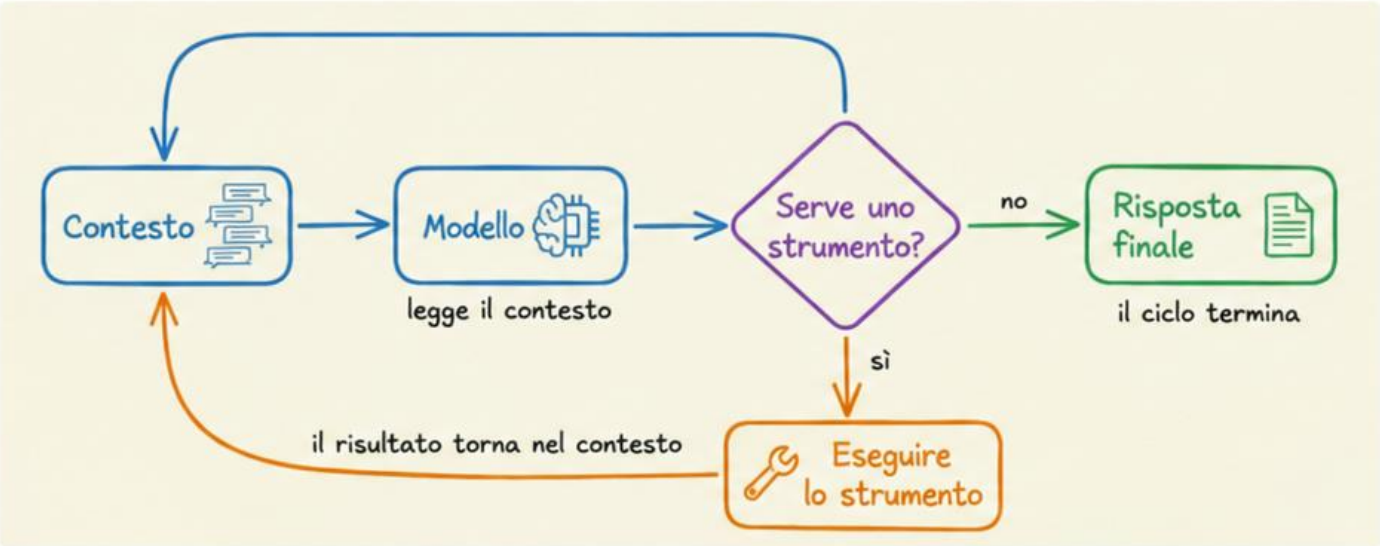
LLM

Ingegneria del prompt, del contesto, dell'harness e del ciclo

Daily Dose of Data Science

3.7.2026

In sostanza, un agente è un ciclo while:



- Il modello viene eseguito
- Richiede chiamate a strumenti
- I risultati degli strumenti tornano nel contesto
- Il modello viene eseguito di nuovo finché non richiede più strumenti

ReAct ha descritto questa forma di ciclo già nel 2022–23, e quasi ogni agente/framework ne esegue un'implementazione simile (**qui abbiamo implementato ReAct da zero in puro Python**).

Ma attorno a questo intero ciclo si articolano quattro livelli di ingegneria:

- Ingegneria del prompt
- Ingegneria del contesto
- Ingegneria dell'harness
- Ingegneria del ciclo

MACHINE LEARNING MASTERY
Making developers awesome at machine learning

maggio 2026

PROMPT ENGINEERING PER L'AI AGENTICA

CHAT VS. PROMPTING AGENTICO

CHAT

UNA BUONA RISPOSTA

AGENTICO

DECISIONI

CHIAMATE A STRUMENTI

PASSAGGI INTERMEDI

GUIDA UN SISTEMA ATTRAVERSO **MOLTI PASSAGGI**



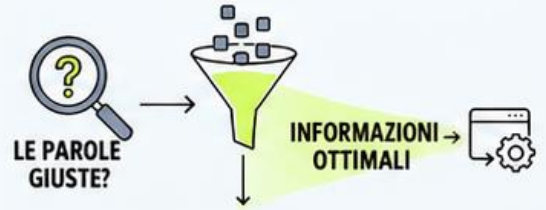
L'AMBIGUITÀ AL PRIMO PASSO PORTA ALLA DERIVA, SENZA UN ERRORE VISIBILE.

DETERIORAMENTO DEL CONTESTO



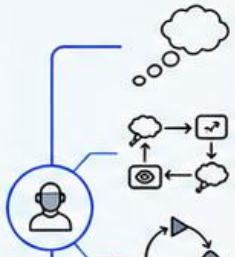
I TOKEN SI ACCUMULANO → PEGGIORANO IL RECUPERO DELLE INFORMAZIONI E LA PRECISIONE DEL RAGIONAMENTO.
OGNI OUTPUT METTE SOTTO PRESSIONE IL BUDGET DI ATTENZIONE.

INGEGNERIA DEL CONTESTO



DALLE «PAROLE GIUSTE?» ALLE «INFORMAZIONI OTTIMALI DI CUI IL MODELLO HA BISOGNO IN OGNI MOMENTO».

ARCHITETTURE DI RAGIONAMENTO



CATENA DI PENSIERO (CoT)
ESPLICITARE I PASSAGGI DI RAGIONAMENTO PRIMA DELL'OUTPUT.

ReAct
(PENSIERO → AZIONE → OSSERVAZIONE → PENSIERO)
BASA IL RAGIONAMENTO SUGLI OUTPUT REALI DEGLI STRUMENTI.

REFLEXION
L'AGENTE VALUTA L'OUTPUT RISPETTO ALL'OBIETTIVO DOPO OGNI PASSAGGIO PRINCIPALE.

SISTEMI MULTIAGENTE
SCHEMA ORCHESTRATORE–ESECUTORI: OGNI ESECUTORE RICEVE SOLO IL CONTESTO NECESSARIO.

ERRORI COMUNI



In questo articolo si vedrà come il prompt engineering cambi radicalmente quando viene applicato ai sistemi di AI agentica, e quali principi e schemi permettano agli agenti di comportarsi in modo affidabile su larga scala.

Gli argomenti trattati includono:

- Perchè formulare prompt per gli agenti è diverso dal farlo per i chatbot e che cosa significa, in pratica, ingegneria del contesto.
- I quattro componenti necessari a ogni prompt per agenti, tra cui *prompt* di sistema, strumenti, esempi e gestione dello stato del contesto.
- Le architetture di ragionamento che rendono gli agenti più affidabili, dalla catena di pensiero a ReAct e *Reflexion*.

Prompt engineering per l'AI agentica

di **Shittu Olumide** il 19 maggio 2026 in **Intelligenza artificiale** 1



Claude Code a 360°

MODELLI ANTHROPIC IN CLAUDE CODE

Quali puoi usare, come vederli e come sceglierli

Sintesi di fonti Anthropic Help Center · aggiornamento 2026

MODELLI DISPONIBILI

La disponibilità esatta dipende dall'account e può cambiare nel tempo

Haiku

`Esempio: claude-haiku-5-5`

Più rapido ed economico.
Utile per ricerche rapide, modifiche semplici e script ripetitivi.

Sonnet

`Esempi: claude-sonnet-5-5 · claude-sonnet-5`

È il default per la maggior parte del lavoro di coding.
Buon equilibrio tra capacità, velocità e costo.

Opus

`Esempi: claude-opus-5-5 · claude-opus-5`

Per problemi più difficili e ragionamento più profondo.
Adatto a refactoring ampi, debug complesso e scelte architetturali.

Fable

`Esempi: claude-fable-5-1 · claude-fable-5`

Può comparire tra i modelli disponibili nel tuo account.
Richiede versioni recenti di Claude Code e può dipendere dal piano.

/model è sempre la fonte di verità per il tuo account. Possono comparire anche versioni precedenti ancora supportate.

COME VEDERLI

1. `/status` — Ti mostra il modello attuale.
2. `/model` — Ti mostra i modelli disponibili per il tuo account e apre il selettore interattivo.

Puoi cambiare modello anche a metà sessione senza perdere la conversazione.

COME SCEGLIERLI

1 Dentro Claude Code

`/model`

Apri il menu e scegli il modello.
Il cambio è immediato.

2 All'avvio della sessione

`claude --model <nome-modello>`

`Esempio: claude --model claude-sonnet-5-5`

Vale per quella sessione.

3 Come predefinito

`ANTHROPIC_MODEL=<nome-modello>`

Imposta il modello preferito per le sessioni future tramite variabile d'ambiente.

GUIDA RAPIDA

- **Sonnet:** scelta consigliata per quasi tutto il coding quotidiano
- **Opus:** usalo quando serve più profondità
- **Haiku:** rapido per compiti semplici o voluminosi
- **Fable:** usalo se il tuo account lo rende disponibile

NOTE IMPORTANTI

- I nomi esatti dei modelli e le versioni cambiano nel tempo
- La disponibilità dipende dall'account e dal metodo di accesso
- Per Fable in Claude Code possono servire build recenti
- Controlla sempre con /model prima di dare per scontata una disponibilità

LIVELLI DI EFFORT IN CLAUDE CODE

Cosa sono, come vederli e come sceglierli

Sintesi di fonti Anthropic · aggiornamento 2026

Effort = quanto lavoro Claude fa per ogni richiesta: letture, tool, verifiche, token e profondità del ragionamento.

1. I LIVELLI DISPONIBILI

Stesso modello, quantità di lavoro diversa

low

Più veloce e parsimonioso.
Meno letture, meno tool, meno verifiche.

Per compiti semplici

medium

Via di mezzo leggera.
Buono per attività di routine e per estendere l'uso.

Routine

high

Più approfondito.
Buon equilibrio tra qualità, velocità e consumo.

Spesso consigliato

xhigh

Più letture e più verifiche.
Utile per codice complesso e lavoro agenziale.

Profondità maggiore

max

Il massimo sforzo.
Per debug difficile e decisioni architetturali importanti.

Consuma di più

auto

Claude usa il livello predefinito del modello.
È il punto di partenza naturale.

Reset al default

Gli stessi nomi non implicano la stessa quantità di lavoro su modelli diversi: se cambi modello, riparti dal default.

2. COME VEDERLI

1. `/effort` — Apre il selettore interattivo e mostra i livelli disponibili.
2. `/status` — Ti mostra il setup attuale. Può aiutarti a controllare modello e configurazione corrente.
3. `intestazione sessione` — Dopo il cambio, il livello attivo compare nella sessione.

Puoi cambiare effort anche a metà sessione.

3. COME SCEGLIERLI

Dentro Claude Code

`/effort`

Usa **/effort** con o senza argomento.
Esempi: `/effort high` · `/effort auto`

All'avvio della sessione

`claude --effort <livello>`

Vale per quella sessione.
Esempio: `claude --effort xhigh`

Come preferenza

`CLAUDE_CODE_EFFORT_LEVEL=<livello>`

Variabile d'ambiente.
Accetta un livello o auto.

Nelle impostazioni

`effortLevel`

Nel file di settings puoi usare: low, medium, high o xhigh.

GUIDA RAPIDA

- Parti dal default del modello
- Alza l'effort se Claude si ferma troppo presto
- Abbassalo per task semplici, rapidi o voluminosi
- Se più effort non basta, valuta un modello più capace

NOTE IMPORTANTI

- Effort non cambia ciò che il modello sa
- Cambia quanto legge, usa tool e verifica
- max è pensato soprattutto per la sessione corrente
- La tua organizzazione può limitare i livelli disponibili
- Ultracode non è un livello di effort

Fonti: Claude Academy "Choosing the right effort level in Claude Code"; Anthropic Help Center; Claude blog "Choosing a Claude model and effort level in Claude Code".

I MODE DI CLAUDE CODE

Sintesi del Markdown · Ottobre 2026

6 PERMISSION MODE | Decidono quali azioni richiedono conferma

`default`

Manual · alias manual

Legge senza conferma.
Per le altre azioni chiede.

`acceptEdits`

Edit automatically

Legge, modifica file nelle cartelle di lavoro e usa comandi filesystem comuni.

`plan`

Plan

Legge, esplora con la shell e prepara un piano.
Modifica il codice dopo l'approvazione.*

`auto`

Auto

Un classificatore valuta le azioni e blocca quelle rischiose.
Le regole ask esplicite chiedono conferma.

Predefinito in terminale e VS Code**

`dontAsk`

Per gli script

Solo letture e strumenti preapprovati.
Nega ciò che richiederebbe conferma.

`bypassPermissions`

Bypass permissions

Esegue senza conferme ordinarie.
Solo container o VM.
Restano deny, ask esplicite e protezioni sui percorsi critici.

COME CAMBIARE MODE

`Shift+Tab` dontAsk escluso dal ciclo

`claude --permission-mode <valore>`

Settings: `permissions.defaultMode`

Precedenza: flag → settings → default

auto e **bypassPermissions:** settings utente o gestiti

ALTRI "MODE": FUNZIONI DIVERSE

AVVIO

`Interactive`	sessione live
`Print`	un prompt, poi esce
`Bare`	print senza personalizzazioni
`Safe`	diagnosi senza personalizzazioni
`Restricted`	blocca comandi e file

PROMPT E RISPOSTE

`Vim`	editing del prompt
`Shell`	comando diretto con !
`Output styles`	tono e formato
`Screen reader`	output lineare

RAGIONAMENTO E VELOCITÀ

`Extended thinking`	ragionamento esteso
`Effort`	intensità del ragionamento
`Ultracode`	script e più subagent
`Fast`	Opus più rapido, costo maggiore

SANDBOX E ATTIVITÀ

`Sandbox`	isolamento dei comandi Bash
`Auto-allow / regular permissions`	
`Background Bash`	attività in background Non è formalmente un mode.
`Ctrl+B · /tasks`	

* **Plan:** nel terminale interattivo con bypass disponibile il blocco delle modifiche non è imposto tecnicamente.

** Per -p e SDK il documento segnala fonti discordanti sul default: verificare la build con `claude --version`.

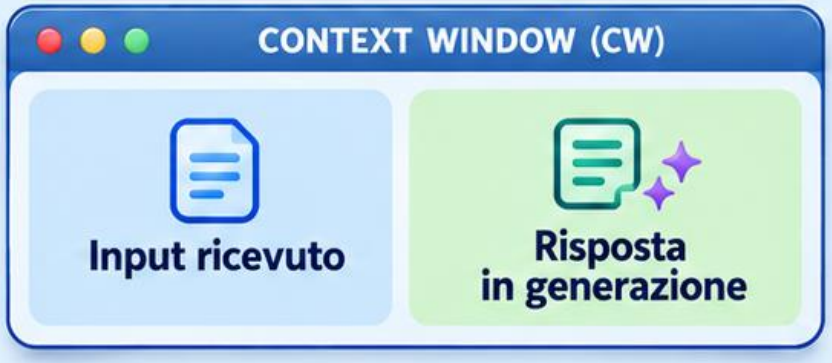
Fonte: "I mode di Claude Code (ottobre 2026)" · @Antonio · Sintesi dell'allegato

CHE COS'È LA CONTEXT WINDOW (CW)

Il contesto di una chiamata e la continuità della conversazione in Claude Code

La CW è lo spazio di lavoro del modello

In una singola chiamata, il modello usa un contesto misurato in token.



L'API Messages è stateless

Il modello non conserva memoria della conversazione tra una chiamata e l'altra.

Claude Code reinvia il contesto corrente:

- definizioni dei tool
- system prompt e CLAUDE.md
- cronologia disponibile + nuovi messaggi

Due chiamate nella stessa sessione

Esempio semplificato: il contesto viene reinviato a ogni chiamata.

CLAUDE CODE
Prepara la richiesta

MODELLO CLAUDE
Genera la risposta

CHIAMATA 1

- Definizioni dei tool
  System prompt • CLAUDE.md
- **Utente:** «Il progetto si chiama Aurora.»

API Messages →

Assistente: «Ricevuto.»

CHIAMATA 2

- Definizioni dei tool
  System prompt • CLAUDE.md
- **Utente:** «Il progetto si chiama Aurora.»
- **Assistente:** «Ricevuto.»
- **Utente:** «Come si chiama il progetto?»

API Messages →

Assistente: «Aurora.»

«Aurora» è presente nel contesto della seconda chiamata.

A ogni chiamata Claude Code reinvia il contesto corrente.
La CW si riempie con nuovi messaggi, risposte e risultati dei tool.

Fonti: Anthropic — Claude Code Docs e Claude API Docs

GUIDA VISIVA · OTTOBRE 2026

La finestra di contesto di Claude

La «memoria di lavoro» dei modelli linguistici: che cos'è, da che cosa dipende la sua dimensione e come la usiamo bene — in chat e in Claude Code.

01 Che cos'è

Tutto ciò che il modello «vede» nel momento in cui genera la risposta: le istruzioni di sistema, i documenti allegati, i messaggi scambiati e la risposta in corso di scrittura. **Ciò che non è nella finestra, per il modello non esiste.**

SI MISURA IN TOKEN — I FRAMMENTI DEL TESTO (␣ = SPAZIO)

`La` `␣fine` `stra` `␣di` `␣conte` `sto`

Un token ≈ 4 caratteri · 1.000 token ≈ 700–750 parole · «500K» e «1M» = 500.000 e 1.000.000 di token.

02 Da che cosa dipende la dimensione

```
finestra = min( modello, tool, piano )
```

MODELLO — il tetto architetturale: il massimo di token che il modello sa gestire.

TOOL — lo stesso modello è servito con finestre diverse a seconda del tool: app Claude (chat), Claude Code, API.

PIANO — Free: limiti variabili con la domanda; a pagamento, alcune finestre da 1M richiedono i crediti di utilizzo (pagamento a consumo).

ESEMPIO — CLAUDE FABLE 5, PIANI A PAGAMENTO



La finestra è una scelta di prodotto, non una costante fisica: servire più contesto costa.

03 Che cosa la occupa



- **Prompt di sistema** — istruzioni di base del prodotto
- **Strumenti e connettori** — caricati anche se non usati
- **Istruzioni di progetto** — rilette a ogni messaggio
- **Documenti e file** — allegati e materiale caricato
- **Conversazione** — messaggi e risposta in corso
- **Spazio libero** — il margine per lavorare

Nei Projects i documenti passano dalla RAG (recupero selettivo): nella finestra entra solo la parte che serve alla domanda, non l'intero archivio.

04 Quando si riempie: la gestione automatica

1. **La chat si avvicina al limite** della finestra di contesto.
2. **Claude riassume i messaggi più vecchi** («sta organizzando i pensieri») per liberare spazio.
3. **La conversazione prosegue** quasi senza limiti, nella maggior parte dei casi.

Piano a pagamento + «Code execution» attiva

La cronologia completa resta consultabile

Il riassunto consuma quota di utilizzo

05 Tre «memorie» da non confondere

Finestra di contesto — il banco di lavoro: ciò che il modello vede adesso. — VIVE IN UNA CHAT

Gestione automatica — gli appunti di sgombero: comprime il passato della stessa chat. — VIVE IN UNA CHAT

Memoria (Memory) — il quaderno nel cassetto: fatti durevoli, a ogni nuova chat. — SEGUE L'ACCOUNT

In Claude Code: **CLAUDE.md** = ciò che scriviamo noi · **auto memory** = ciò che scrive Claude · `/memory` le gestisce entrambe

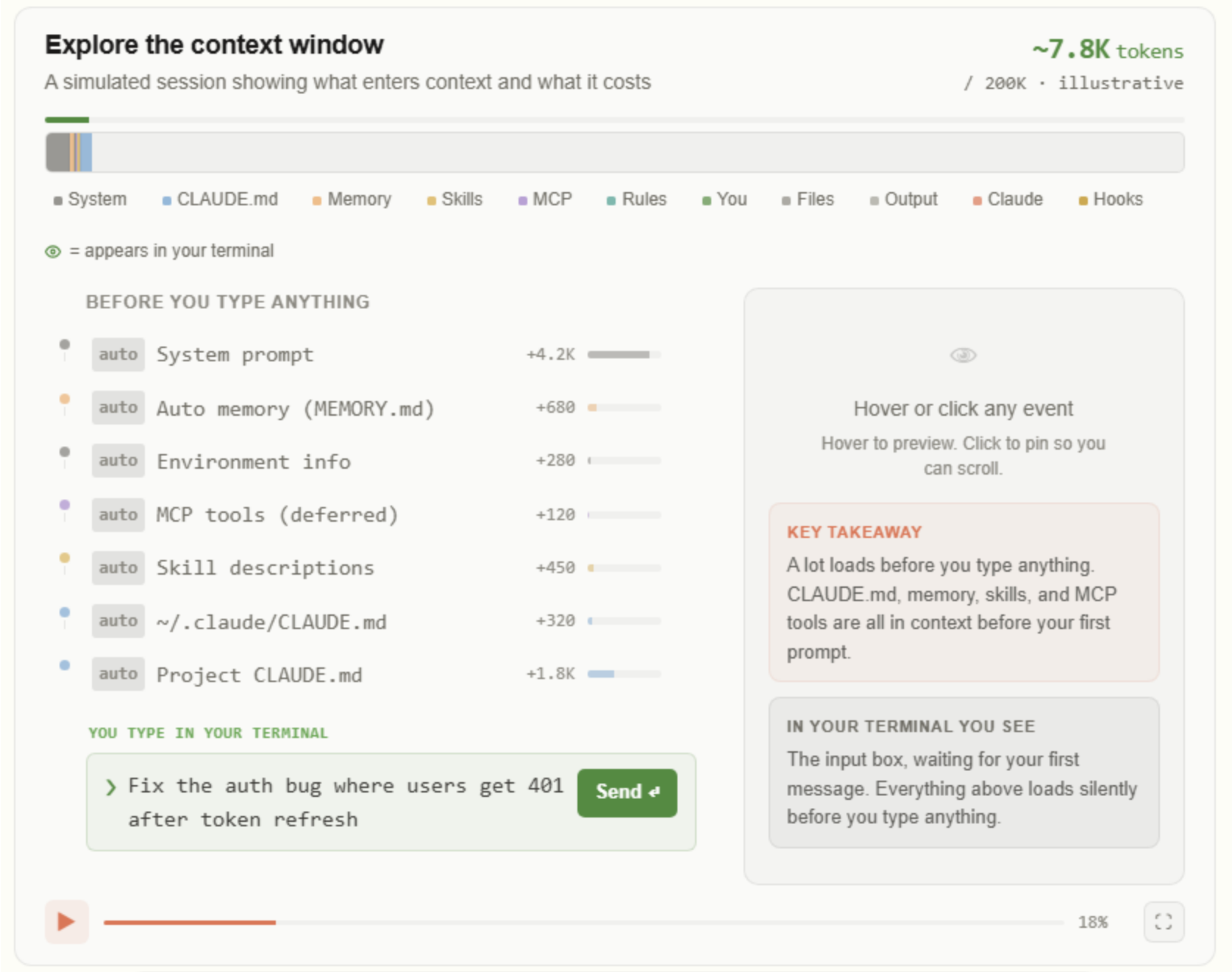
06 Regole pratiche (e il «context rot»)

Il degrado da contesto è reale; le soglie fisse (20%, 40%…) no: **conta che cosa c'è nella finestra, non solo quanto è piena.**

`/clear`	Quando cambiamo compito: il contesto precedente è rumore.
`/compact`	Deliberato, ai punti di svolta: mai aspettare quello automatico vicino al limite.
`Igiene`	Pochi connettori attivi, istruzioni concise, documenti nei Projects.
`Sintomi`	Compattiamo quando il modello rilegge file già letti, reintroduce bug corretti o dimentica decisioni prese.

Fonti: support.claude.com («How large is the context window on paid Claude plans?») · code.claude.com/docs · Ricerca: Lost in the Middle (Liu et al., 2024), Context Rot (Chroma Research). Infografica didattica — ottobre 2026.

link alla timeline interattiva



la dimensione della finestra di contesto dipende dunque non solo dal modello, dal tool ma anche dal piano??

Cercato sul web, letta una pagina ›

Sì, esatto — anche se con una precisazione: tra i vari piani *a pagamento* le dimensioni sono quasi sempre identiche; il piano incide soprattutto ai margini. La dimensione effettiva della finestra di contesto è determinata dall'incrocio di tre fattori:

1. Il modello — il limite architetturale: quanti token il modello è stato addestrato e ottimizzato a gestire (es. 1M per Fable 5.1, 500K per Fable 5 in chat). Oltre questo non si va mai.

2. La superficie d'uso (il "tool") — lo stesso modello viene servito con finestre diverse a seconda del prodotto: le tabelle che hai fotografato mostrano Fable 5 a 500K in chat ma 1M in Claude Code. Anche l'API ha le sue regole (per alcuni modelli la finestra da 1M è un'opzione da richiedere esplicitamente). E persino la piattaforma cloud conta: lo stesso modello su un cloud di terze parti può avere una finestra ridotta rispetto all'API diretta di Anthropic.

3. Il piano — qui gli effetti sono tre:

- **Free vs a pagamento**: la differenza grande. L'articolo di supporto sulla lunghezza massima dei prompt dice esplicitamente che sul piano gratuito i limiti **variano con la domanda** — nei momenti di picco la finestra disponibile può ridursi. Le tabelle da 500K/1M valgono solo per i piani a pagamento.
- **Tra i piani a pagamento** (Pro, Max, Team, Enterprise): le dimensioni nominali sono le stesse, ma cambiano le **condizioni di accesso**. Esempi dalle tue stesse tabelle: in Claude Code, gli utenti Pro devono abilitare i crediti di utilizzo (usage credits, il pagamento a consumo oltre la quota del piano) per avere la finestra da 1M sui modelli Opus; per Sonnet 4.6 i crediti servono su tutti i piani tranne gli Enterprise a consumo. Quindi la finestra "esiste" per tutti, ma su alcuni piani è dietro una condizione di fatturazione.
- **Sull'API**: l'accesso alla finestra da 1M su alcuni modelli dipende dal *usage tier* (il livello di spesa dell'account) — anche qui, stessa logica: il limite tecnico è del modello, ma chi può usarlo per intero lo decide il piano.

Quindi la formula completa per un corso sarebbe: **finestra effettiva = min(limite del modello, configurazione della superficie, ciò che il piano/fatturazione sblocca)** — più la variabile "domanda" sul piano gratuito. È un buon esempio del fatto che la finestra di contesto non è una costante fisica del modello ma una scelta di prodotto: servire 1M di token a ogni richiesta costa (memoria GPU e calcolo crescono con la lunghezza del contesto), quindi chi paga di più, o paga a consumo, ottiene la configurazione più generosa.

link alla pagina della documentazione di Claude

Claude Code Docs

Ottobre 2026

How large is the context window on paid Claude plans?

Updated over a week ago

Copy for LLM

This article explains how large the context window is on paid Claude plans (Pro, Max, Team, Enterprise) when you chat with Claude, or use Claude Code or Claude Cowork.

Chatting with Claude

Model	Context window
Claude Fable 5.1	1M tokens
Claude Fable 5	500K tokens
Claude Opus 5.5	1M tokens
Claude Opus 5	1M tokens
Claude Opus 4.8	500K tokens
Claude Opus 4.7	500K tokens
Claude Opus 4.6	500K tokens
Claude Sonnet 5.5	1M tokens
Claude Sonnet 5	1M tokens
Claude Sonnet 4.6	500K tokens
Claude Haiku 5.5	1M tokens

Claude Code

Claude Code Docs

Ottobre 2026

Model	Context window
Claude Fable 5.1	1M tokens
Claude Fable 5	1M tokens
Claude Opus 5.5	1M tokens
Claude Opus 5	1M tokens
Claude Opus 4.8	1M tokens
Claude Opus 4.7	1M tokens
Claude Opus 4.6	1M tokens **Note:** 1M context window available by selecting `claude-opus-4-6[1m]` with `/model`; on Pro, usage credits must be enabled to access
Claude Sonnet 5.5	1M tokens
Claude Sonnet 5	1M tokens
Claude Sonnet 4.6	1M tokens **Note:** 1M context window available by selecting `claude-sonnet-4-6[1m]` with `/model`; usage credits must be enabled to access (except for usage-based Enterprise plans)
Claude Haiku 5.5	1M tokens

Le due tabelle (la slide precedente e questa) non sono contraddittorie: descrivono due prodotti diversi. La prima tabella vale per le chat nell'app Claude (web/desktop/mobile), la seconda per Claude Code, e **lo stesso modello può avere una finestra di contesto diversa a seconda di dove viene usato**.

In sintesi, dall'articolo ufficiale:

- **Nell'app Claude (chat)**: Opus 5 e Sonnet 5 supportano una finestra di contesto da 1M di token su tutti i piani a pagamento, mentre Opus 4.8, 4.7, 4.6 e Sonnet 4.6 si fermano a 500K — da qui i 500K che vedi anche per Fable 5 nella prima tabella. claude
- **In Claude Code**: con un piano Pro, Max, Team o Enterprise, Sonnet 5, Fable 5, Opus 5, 4.8, 4.7 e 4.6 supportano tutti 1M di token; gli utenti Pro devono però abilitare i crediti di utilizzo per avere la finestra da 1M sui modelli Opus, e Sonnet 4.6 richiede i crediti di utilizzo su tutti i piani tranne gli Enterprise a consumo. claude

COSA CONTIENE LA CONTEXT WINDOW (CW)

Cosa entra all'avvio e cosa si aggiunge durante il lavoro in Claude Code

GIÀ ALL'AVVIO

Istruzioni di base
System prompt e informazioni sull'ambiente.

CLAUDE.md globale e di progetto
`Globale: ~/.claude/CLAUDE.md`
`Progetto: ./CLAUDE.md o ./.claude/CLAUDE.md`

Auto memory
Parte iniziale di MEMORY.md, se presente.

Skill, tool nativi e tool MCP
Descrizioni delle skill, definizioni dei tool nativi e nomi dei tool MCP.
Con Tool Search, le definizioni MCP complete arrivano su richiesta.

7.850 token già prima del primo prompt
Nella simulazione ufficiale; il valore reale varia con la configurazione.

DURANTE IL LAVORO

Le letture dei file spesso dominano
Ogni Read aggiunge alla CW tutto il contenuto restituito, anche se nel terminale si vede solo una riga.
Il file intero, oppure la porzione richiesta.

Conversazione
Prompt, risposte e chiamate ai tool.

Risultati dei tool, anche MCP
Ricerche, test, errori, log e altri output.

Contesto aggiunto quando serve
Istruzioni delle skill usate, regole pertinenti e contesto degli hook.

COME SI RIEMPIE

Schema illustrativo

Ogni nuovo contenuto aggiunge token.



`/context` → mostra l'occupazione per categoria

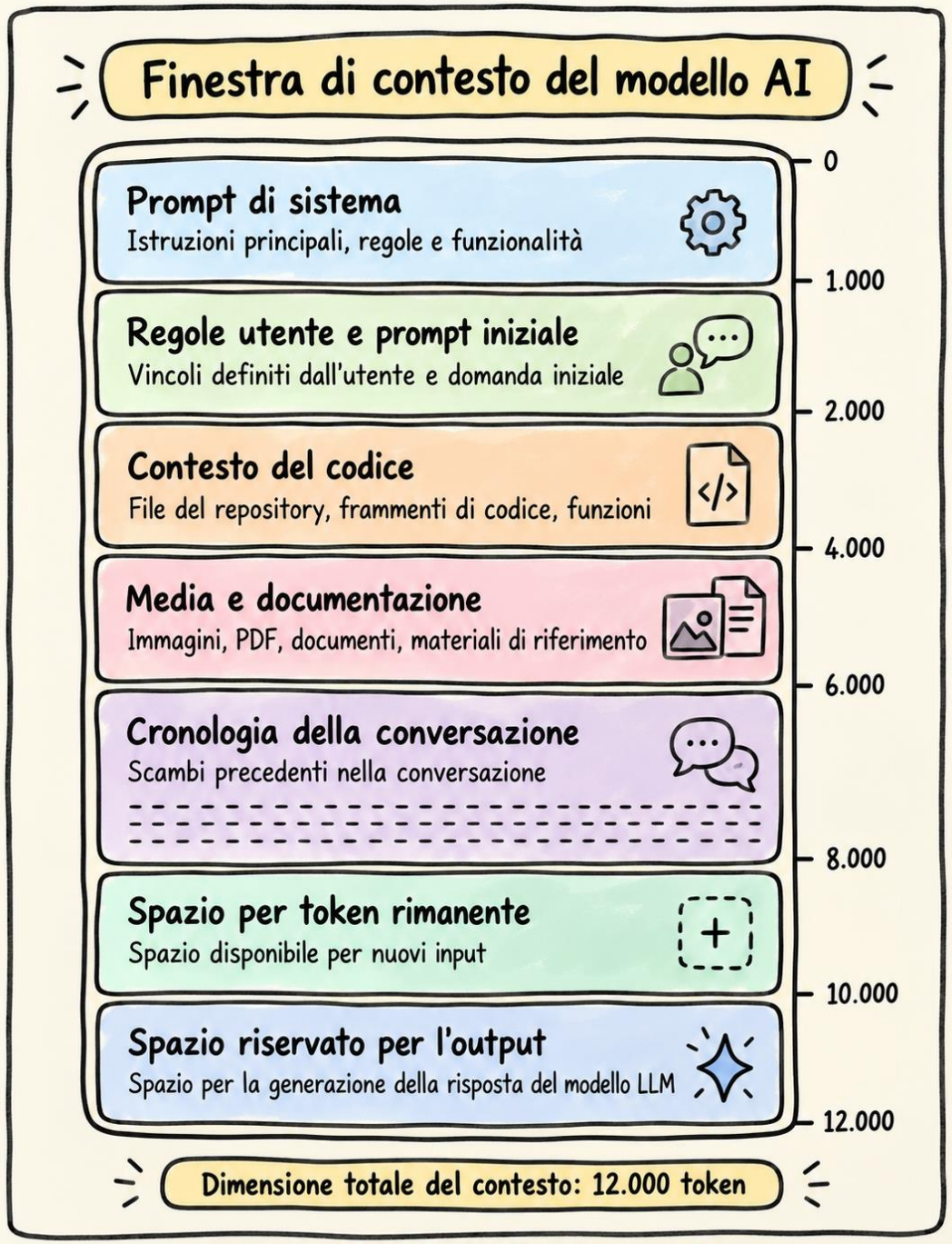
IL TRUCCO DEI SUBAGENT

Una CW separata: carica tool MCP e, quando previsti, CLAUDE.md.

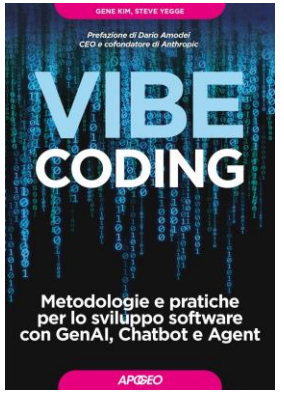


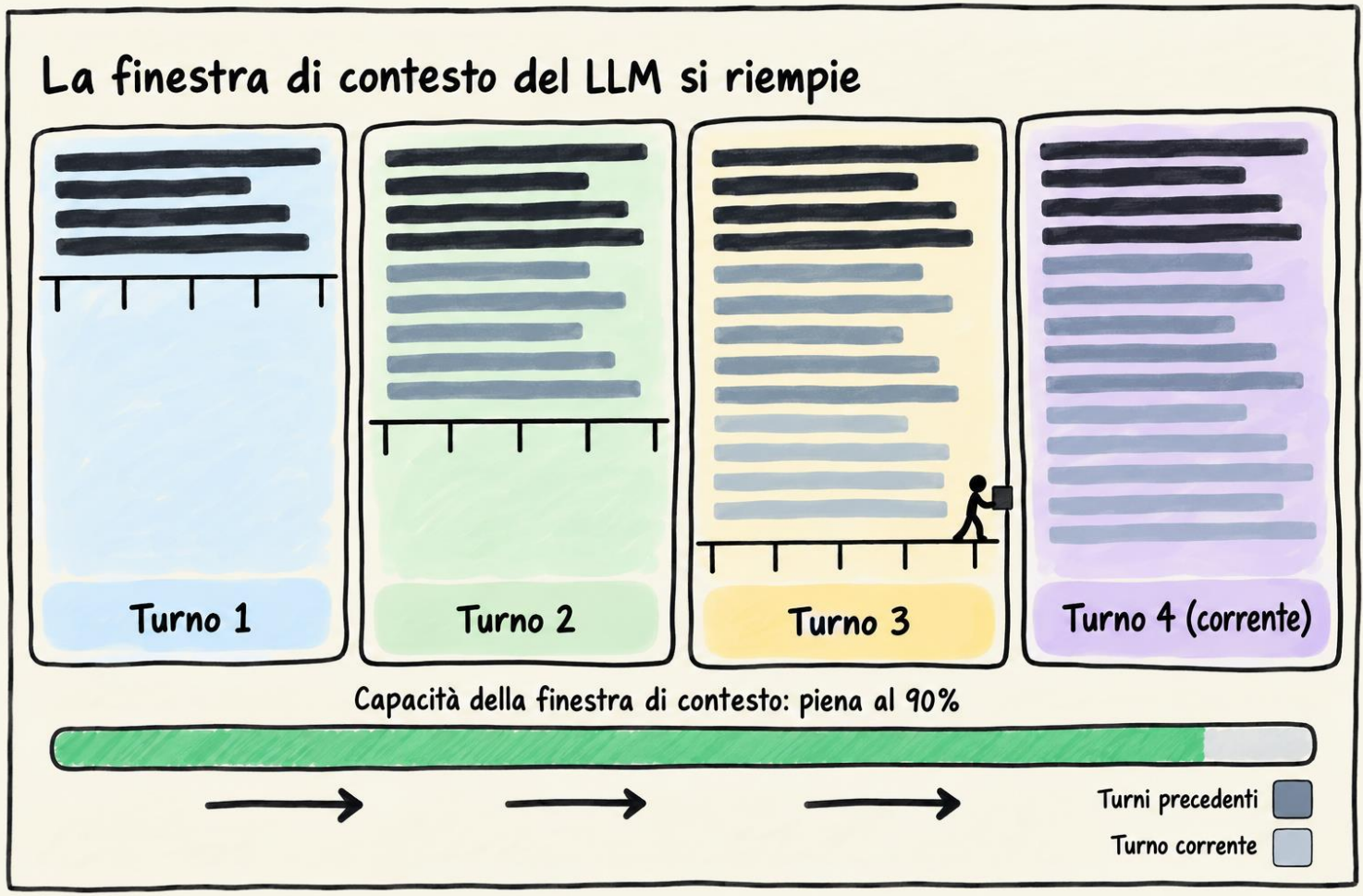
Al principale torna solo la risposta finale + un piccolo trailer di metadati.
Esempio dei docs. I CLAUDE.md caricati dipendono dal tipo e dalla configurazione.

Fonti: Anthropic — Claude Code Docs e simulazione ufficiale



Schema di una finestra di contesto di un modello AI.





COMPATTAZIONE DELLA CONTEXT WINDOW (CW)

Un riassunto per continuare la sessione: funzionamento, controlli e rischi in Claude Code

CHE COS'È

Claude Code riassume la cronologia e usa la sintesi nel contesto attivo, liberando token per proseguire.



Schema semplificato

La sessione continua con meno cronologia dettagliata nella CW.

AUTOMATICA, MA DISATTIVABILE

Attiva di default; la soglia dipende dal modello e dalla configurazione.



Con OFF: usare /compact manualmente prima del limite.

IL RISCHIO

La CW può raggiungere il limite: le nuove richieste si fermano.

```
>_ Context limit reached
```

Per continuare: **/compact** oppure **/clear**.

CHE COSA RESTA E COSA PUÒ PERDERSI

Nel riassunto
Cerca di conservare obiettivi, decisioni, stato del lavoro e attività aperte.

Istruzioni persistenti
System prompt ancora valido; CLAUDE.md principali e auto memory ricaricati.

Possibili perdite
Dettagli, vecchi output e istruzioni presenti solo nella cronologia possono sfuggire.

Per guidare anche la sintsi automatica: sezione «Compact Instructions» in CLAUDE.md.

COMANDI PER GESTIRLA

`/compact [istruzioni]`	Riassume subito; le istruzioni indicano le priorità. `/compact conserva decisioni, vincoli e attività aperte`
`/autocompact`	Regola la soglia automatica (v2.1.221+). `L'attivazione ON/OFF si gestisce in /config.`
`/rewind → Summarize…`	Riassume una parte della conversazione.
`/clear`	Avvia una nuova conversazione.
`/context`	Mostra l'occupazione della CW.

Fonti: Anthropic — Claude Code Docs

Come leggere l'output di /context in Claude Code

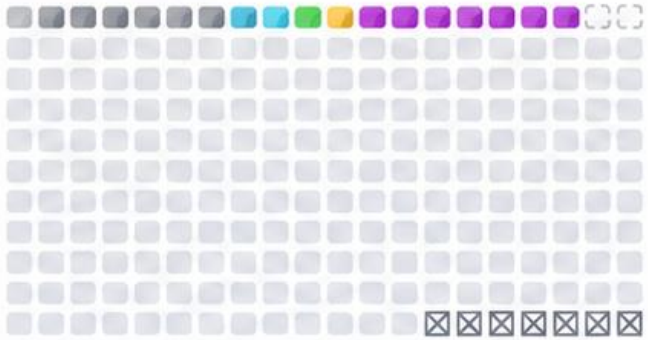
Guida rapida alla lettura del consumo di contesto, dei token e delle componenti della sessione

Più consapevolezza del contesto, conversazioni migliori

Output di esempio: /context

Output reale semplificato e formattato

```
/context
└ Context Usage
```



```
Auto-compact window: 1m tokens

MCP tools · /mcp (loaded on-demand)
54 tools · 644 tokens
```

Opus 5.5
`claude-opus-5-5`
`79.3k/1m tokens (8%)`

Estimated usage by category

- System prompt: 2k tokens (0.2%)
- System tools: 36.9k tokens (3.7%)
- MCP tools: 644 tokens (0.1%)
- MCP server instructions: 826 tokens (0.1%)
- Skills: 2.8k tokens (0.3%)
- Messages: 36.2k tokens (3.6%)
- Free space: 887.7k tokens (88.8%)
- Autocompact buffer: 33k tokens (3.3%)

Che cosa significa

Model / finestra di contesto	Indica il modello attivo e la capienza massima disponibile.
79.3k / 1m tokens	Quanti token della finestra sono già occupati in questa sessione.
8%	Percentuale di contesto usata finora.
Estimated usage by category	Scomposizione stimata del consumo per tipologia di contenuto.
Free space	Spazio ancora disponibile prima di saturare la finestra.
Autocompact buffer	Margine riservato per permettere il compact automatico prima di arrivare al limite.
Auto-compact window	Soglia/ampiezza usata dal meccanismo di compattazione del contesto.
MCP tools	Strumenti esterni disponibili viaa MCP; qui sono caricati on-demand.

Come interpretare le categorie

System prompt	Istruzioni di base del sistema.
System tools	Descrizioni degli strumenti interni disponibili al modello.
MCP tools	Descrizioni sintetiche dei tool MCP esposti.
MCP server instructions	Istruzioni aggiuntive provenienti dai server MCP.
Skills	Skill caricate o disponibili.
Messages	Cronologia della conversazione e contenuti scambiati.

Lettura pratica dell'esempio

1. La sessione è molto lontana dal limite: è usato solo l'**8%** del contesto.
2. I contributi principali qui sono `System tools` e **Messages**
3. Lo spazio libero è ancora molto ampio (**88.8%**).
4. L'autocompact aiuta a evitare saturazione e perdita improvvisa di contesto.
5. I tool MCP ci sono, ma il loro peso attuale è minimo (**644 token**).

Suggerimento hands-on: controlla `/context` durante attività lunghe o con molti tool per capire quando il contesto sta crescendo troppo.

Meno sorprese, più produttività

LIMITI E DEGRADAZIONE DELLA CONTEXT WINDOW (CW)

Quanto contesto entra e quanto bene Claude riesce a usarlo

CAPIENZA E AFFIDABILITÀ



Capienza nominale

È finita e dipende dal modello e dalla configurazione.

Esempi di capienza

200K token **1M token**



Uso affidabile del contesto

Recuperare dettagli, collegare informazioni lontane e rispettare i vincoli.

La qualità può calare prima che la CW sia piena.

IL DEGRADO: CONTEXT ROT

La qualità del recupero e del ragionamento può diminuire con l'aumento del contesto.

Il rumore può amplificare il problema.

- **Testo irrilevante**
- **Log ridondanti**
- **Istruzioni contrastanti**

Possibili segnali da verificare

- **Vincoli trascurati**
- **Dettagli confusi**
- **Tentativi falliti ripetuti**

L'effetto dipende da modello, compito e contenuti.

LOST IN THE MIDDLE

Schema illustrativo

Anche la posizione dell'informazione può influire.



In alcuni test, i dati al centro vengono recuperati peggio di quelli all'inizio o alla fine.

Effetto osservato in specifici test, non una regola universale.

COME RIDURRE IL RISCHIO



Separare i compiti
Evitare di mescolare attività non correlate.



Selezionare il contesto
File pertinenti e output essenziali, mantenendo le informazioni necessarie.



Rendere chiari i vincoli
Obiettivo, stato attuale e decisioni valide devono essere espliciti.



Verificare il risultato
Usare test e riscontri sui file correnti.



Un buon contesto contiene tutte le informazioni necessarie, con poco rumore.

Fonti: Anthropic · Liu et al., Lost in the Middle · Chroma, Context Rot

LIMITI E DEGRADAZIONE DELLA CONTEXT WINDOW (CW)

Capienza, qualità e gestione del contesto in Claude Code

LIMITI, RUMORE E POSIZIONE

Capienza nominale:
finita; dipende da modello e configurazione. Esempi: 200K o 1M token.

Context rot:
aumentando il contesto, recupero e ragionamento possono peggiorare. Testi irrilevanti, log ridondanti e istruzioni contrastanti aumentano il rumore.

Possibili segnali:
vincoli trascurati, dettagli confusi, tentativi falliti ripetuti.



Lost in the middle:
in alcuni test, i dati al centro sono recuperati peggio. Non è una regola universale.

Selezionare contenuti pertinenti e verificare sui file correnti.

REGOLARE L'AUTOMATISMO

Claude Code ≥ 2.1.221

`/autocompact 500k` Imposta una finestra di auto-compattazione di 500K token.

- Valori ammessi: 100K–1M token.
- Formati: 500000, 500k, 1M.
- Numeri da 100 a 1000 = migliaia: 500 = 500K.

Il valore effettivo è limitato dalla CW del modello:
500k non amplia una CW da 200K.

`/autocompact auto` Ripristina la finestra prevista per il modello.

ON/OFF: `/config` → **Auto-compact**

COMPATTARE O RIPARTIRE

`/compact focus on <cosa>`
Orienta il riassunto sulle informazioni da conservare.

Esempio: `/compact focus on bug di login e test`
focus on è un'istruzione libera, non un'opzione speciale.

`/clear` Per un nuovo lavoro non correlato: ripartire puliti, senza trascinare un riassunto approssimativo.

COMPATTARE A UN CONFINE PULITO
Quando possibile, a task concluso e non a metà del debug.
Se occorre compattare durante il debug, fissare prima ipotesi, prove, risultati e prossimo passo.

COSA SOPRAVVIVE DOPO /compact

- **Cronologia** → riassunto; dettagli e output integrali possono perdersi.
- **Restano validi** → system prompt e output style.
- **Ricaricati** → CLAUDE.md globale/progetto, regole generali, auto memory e piano. Git aggiornato, se presente.
- **Su richiesta** → regole sui percorsi e CLAUDE.md annidati.
- **File** → fino a 5 riletti, priorità ai modificati più recenti; oltre 5K token, solo riferimento.
- **Skill usate** → max 5K token/skill, 25K totali; le più vecchie possono uscire.
- **Hook** → contesto precedente riassunto; SessionStart(compact) rieseguiti.
- **Background** → comandi e subagent continuano.

Più finestra non garantisce più qualità.
200K token curati possono funzionare meglio di 1M lasciati accumulare.

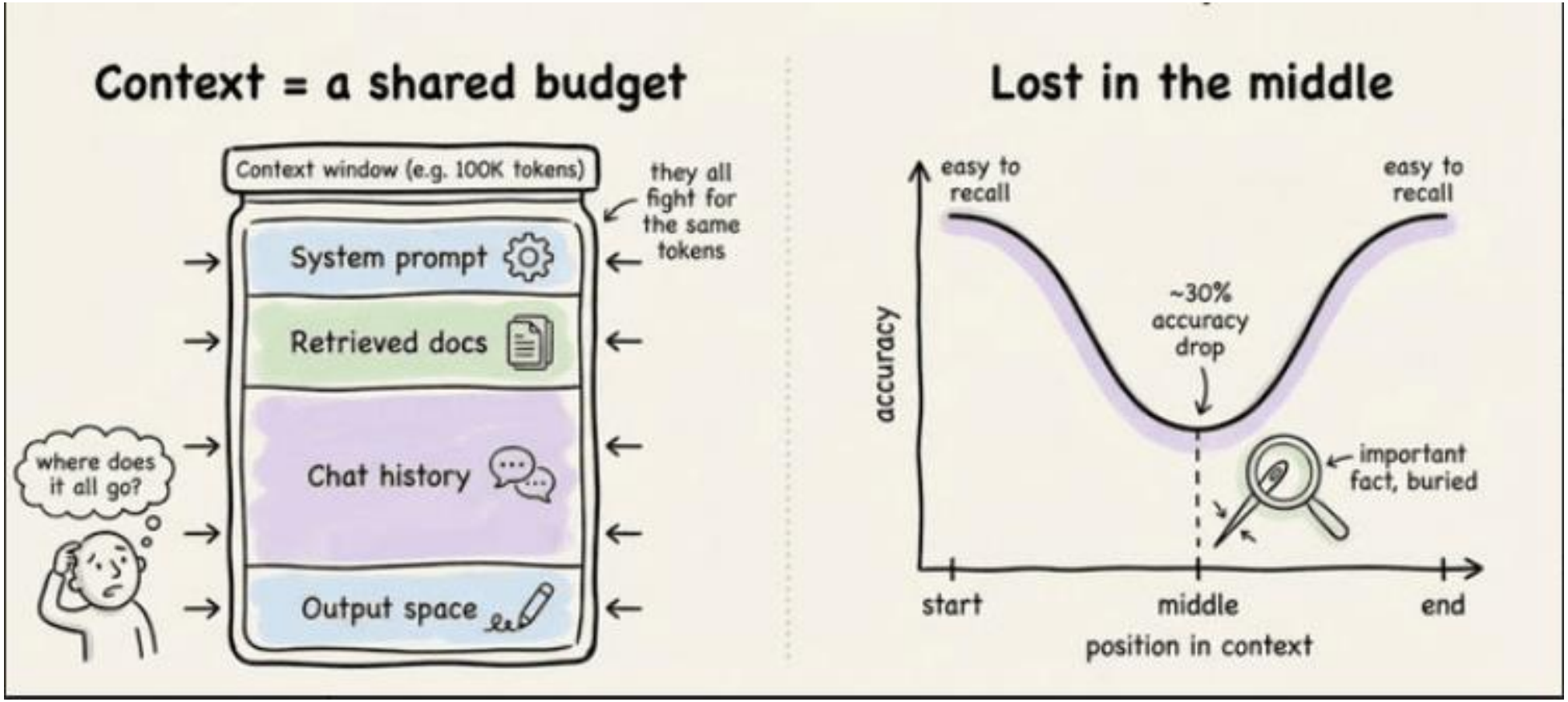
Confronto illustrativo: servono tutte le informazioni pertinenti, con poco rumore.

Fonti: Claude Code Docs — model-config, context-window, commands, prompt-caching · Anthropic · Liu et al., Lost in the Middle · Chroma, Context Rot

Il noto problema «lost in the middle» del contesto

Daily Dose of Data Science

24.04. 2026



A bigger window is not the same as better memory. It just gives the model more room to miss things.

Vedi anche questo articolo

ByteByteGo Newsletter

Automatic context management

For users on paid plans with code execution enabled, Claude automatically manages your conversation context. When your conversation approaches the context window limit, Claude summarizes earlier messages to make room for new content. This allows conversations to continue indefinitely in most cases. Longer conversations that trigger automatic context management use more of your usage limit.

Your full chat history is preserved so Claude can reference it, even after earlier portions have been summarized. You may occasionally notice Claude "organizing its thoughts" during long conversations—this is the automatic context management at work.

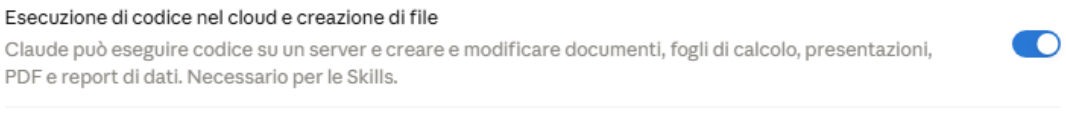
Note: Code execution must be enabled for automatic context management to work. In rare edge cases (such as very large first messages or system errors), you may still encounter context window limits.

Si trova nelle impostazioni dell'app Claude, e il nome esatto dell'interruttore è **"Code execution and file creation"** ("Esecuzione di codice e creazione di file"). Il percorso dipende dal tipo di piano:

- **Piani Free, Pro e Max** (il caso individuale): **Settings > Capabilities** (Impostazioni > Funzionalità), dove si attiva o disattiva l'interruttore **Code execution and file creation**. Dal web il percorso diretto è claude.ai → icona del profilo in basso a sinistra → Settings → Capabilities. claude
- **Piani Team ed Enterprise**: la decisione spetta ai proprietari dell'organizzazione, che la regolano in **Organization settings > Capabilities** con lo stesso interruttore; per i singoli utenti dell'organizzazione la voce può quindi non essere modificabile. claude

Due note utili:

- La funzionalità è disponibile per tutti gli utenti Claude (Free, Pro, Max, Team ed Enterprise) su web, Claude Desktop e Claude Mobile, ed è attiva per impostazione predefinita — quindi è molto probabile che sul tuo account sia già accesa senza che tu debba fare nulla. claude
- Attenzione a non confonderla con Claude Code: nonostante il nome simile, questo interruttore riguarda solo la sandbox (l'ambiente di esecuzione isolato) delle chat nell'app Claude — quella che abilita la creazione di file e, come visto prima, la gestione automatica del contesto. Claude Code, lo strumento da terminale, è un prodotto separato e non è controllato da questa impostazione.



Automatic context management (gestione automatica del contesto)

È il meccanismo con cui l'app Claude evita che una conversazione lunga "sbatta" contro il limite della finestra di contesto (la quantità massima di token — unità di testo — che il modello può tenere in vista in un dato momento, i 500K o 1M delle tabelle precedenti).

Funziona così:

- **Quando si attiva**: solo su piani a pagamento e solo se l'esecuzione di codice (*code execution*, la funzione che permette a Claude di eseguire codice in un ambiente sandbox) è abilitata. Il riquadro "Note" lo ribadisce: senza code execution, la gestione automatica non funziona.
- **Cosa fa**: quando la conversazione si avvicina al limite della finestra, Claude **riassume i messaggi più vecchi** per liberare spazio. I riassunti occupano molti meno token degli originali, quindi la conversazione può proseguire praticamente all'infinito nella maggior parte dei casi.
- **La cronologia non si perde**: l'intera chat resta salvata e Claude può andarla a riconsultare anche dopo che le parti iniziali sono state riassunte. Nel contesto attivo c'è il riassunto, ma l'originale rimane recuperabile.
- **L'indizio visibile**: durante le conversazioni lunghe si può notare Claude che "organizza i pensieri" (*organizing its thoughts*) — quella pausa è proprio il riassunto in corso.
- **Il costo nascosto**: le conversazioni lunghe che attivano questo meccanismo **consumano più limite di utilizzo** (*usage limit*, la quota di messaggi/token del piano), perché l'operazione di riassumere è a sua volta lavoro del modello.
- **I casi limite**: in situazioni rare — un primo messaggio enorme (es. un documento gigantesco incollato subito), o errori di sistema — si può comunque incontrare il limite, perché non c'è "materiale vecchio" da riassumere.

Maximizing your context window

While context is managed automatically for most conversations, you can still optimize how you use your available context space:

- **Utilize projects effectively:** Projects use retrieval-augmented generation (RAG), which allows Claude to work with larger amounts of information by only loading relevant content into the context window.
- **Keep project instructions concise:** Claude performs best when you use project instructions for general context around your project, key guidelines, and Claude's role.
- **Manage tools and connectors:** These features are token-intensive, so being mindful of how many you have active helps maximize your available context.

È la sezione "Maximizing your context window" ("Ottimizzare la finestra di contesto") che avevamo già incrociato nell'immagine precedente — qui la vediamo per intero. Il messaggio di apertura dice: anche se per la maggior parte delle conversazioni il contesto viene gestito automaticamente (il riassunto automatico di cui parlavamo), conviene comunque usare bene lo spazio disponibile. Seguono tre consigli pratici:

1. Utilize projects effectively (usare bene i Projects)

I Projects sono le aree di lavoro di claude.ai in cui si caricano documenti di riferimento (la "knowledge base" del progetto). Il punto chiave è che i Projects usano la RAG, *Retrieval-Augmented Generation*: una tecnica in cui i documenti non vengono caricati per intero nel contesto a ogni messaggio, ma vengono prima indicizzati, e poi a ogni domanda viene recuperata e inserita nel contesto solo la parte rilevante. Il vantaggio: si può lavorare con una mole di materiale molto più grande della finestra di contesto stessa, perché in finestra entra solo ciò che serve in quel momento. (È esattamente il meccanismo che tratti nei tuoi corsi sulla RAG, applicato qui a livello di prodotto.)

2. Keep project instructions concise (istruzioni di progetto concise)

Le *project instructions* sono le indicazioni permanenti che si scrivono in un Project ("sei un revisore di testi, rispondi in italiano, usa questo formato…"). A differenza dei documenti, queste **vengono caricate integralmente nel contesto a ogni messaggio**, senza passare dalla RAG. Quindi ogni parola in più è un costo fisso che si paga sempre. Il consiglio: usarle solo per il contesto generale, le linee guida chiave e il ruolo di Claude — i contenuti lunghi vanno invece nei documenti del Project, dove la RAG li gestisce in modo selettivo.

3. Manage tools and connectors (gestire strumenti e connettori)

I connettori sono le integrazioni con app esterne (Gmail, Google Drive, Slack…) e gli strumenti sono le capacità aggiuntive (ricerca web, ecc.). Ognuno di essi, quando è attivo, inserisce nel contesto le proprie definizioni — la descrizione di cosa fa e di come va chiamato — **anche se nella conversazione non lo si usa mai**. L'articolo li definisce "token-intensive", cioè avidi di token. Il consiglio: tenere attivi solo i connettori che servono davvero, disattivando gli altri, così da liberare spazio per il contenuto vero della conversazione.

In sintesi, i tre punti rispondono alla stessa domanda da tre angolazioni: *cosa occupa la finestra di contesto oltre ai messaggi?* I documenti (→ delegarli alla RAG dei Projects), le istruzioni fisse (→ tenerle brevi) e le definizioni degli strumenti (→ attivarne pochi).

è vera questa regola del pollice: oltre il 20% di occupazione della finestra di contesto conviene fare nuova chat (o /clear in claude code) oppure compattare la chat?

Lette 2 pagine, cercato sul web >

In parte. Il fenomeno sottostante è reale e ben documentato, ma la soglia del 20% come regola universale non è sostenuta né dalla documentazione ufficiale né dalla ricerca: è una euristica di community, e peraltro una delle più aggressive in circolazione. Vediamo cosa dice l'evidenza.

Il fenomeno è vero: il "context rot" (degrado da contesto). Riempire la finestra di contesto non è gratis: il modello non diventa "stanco", ma peggiora nell'*usare* le informazioni che già ha. Diversi studi lo misurano:

- **Lost in the Middle** (Liu et al., 2024): i modelli recuperano bene le informazioni all'inizio e alla fine del contesto, male quelle in mezzo — in un test con 20 documenti, l'accuratezza sulle informazioni centrali scendeva al 25% contro il 42% di quelle finali.
- **Context Rot** (Chroma Research): su 18 modelli testati, le prestazioni calavano di oltre il 30% quando l'informazione rilevante stava in mezzo al contesto, e i "distrattori" semanticamente simili peggioravano le cose.
- Altri lavori (Du et al., 2025) mostrano cali del 14-85% al crescere della lunghezza dell'input, *anche quando il modello recuperava tutte le informazioni rilevanti* — perfino sostituendo il contesto irrilevante con spazi bianchi il calo restava.

Ma le soglie numeriche sono deboli. In giro si trovano il 20%, il 40% (la "regola del 40%", che distingue una "smart zone" da una "dumb zone"), il 70%... Il punto debole: gran parte della ricerca citata usa modelli piccoli (es. Qwen 7B, dove la soglia critica era al 40-50%), dove l'effetto è molto più marcato; sui modelli di frontiera il degrado è più dolce, senza un "precipizio" a una percentuale precisa. Lo ammettono gli stessi autori di queste regole del pollice. Il 20% in particolare viene dall'esperienza personale di singoli sviluppatori, senza benchmark a supporto.

La regola migliore non è una percentuale, ma un criterio di pertinenza. Il consiglio ufficiale di Anthropic per Claude Code è usare `/clear` *tra un compito e l'altro*, non a una soglia fissa: quando cambiamo argomento, tutto il contesto precedente è diventato rumore — distrattori semanticamente simili, che come visto sopra sono i peggiori. Riformulando:

- **Compito nuovo e scollegato** → `/clear` (o nuova chat), anche se siamo al 10% di occupazione
- **Stesso compito, contesto che si allunga** → si può lavorare tranquillamente ben oltre il 20%; `/compact` quando compaiono i sintomi tipici (il modello rilegge file già letti, reintroduce bug già corretti, dimentica decisioni prese nella stessa sessione) o prima di una fase delicata
- **Mai arrivare alla compattazione automatica** sotto pressione (scatta vicino al limite): a quel punto il riassunto butta via dettagli che non scegliamo noi — meglio compattare deliberatamente a un punto di svolta naturale del lavoro, dove sappiamo cosa serve conservare

In sintesi: la direzione della regola è giusta (meno contesto irrilevante = risposte migliori, e molto prima del limite nominale), la soglia fissa del 20% no — è troppo prudente come regola generale e comunque il *quando* dipende più da *cosa* c'è nella finestra che da *quanto* è piena.

Sources: Why I compact my AI coding agent at 20%, not 90% — Level Up Coding, The context 40 percent rule — deknijf.com, Claude Code context management — SitePoint

Incide anche il «workload» del momento? Ufficialmente, no; in realtà, sì.

Come interpretare l'output di /memory in Claude Code

Guida pratica alla schermata di memoria, istruzioni e auto-memory

Claude Code
PIÙ CONTESTO
PER LAVORARE MEGLIO

Output di esempio

Esempio semplificato dell'output di /memory

```
> /memory
  └ Opened ~/.claude/CLAUDE.md
    | To use a different editor, set the $EDITOR or $VISUAL environment variable.

> /memory
  └ Cancelled memory editing

 Memory

> Auto-memory   true

> User instructions      Saved in ~/.claude/CLAUDE.md
  Project instructions   Saved in ./CLAUDE.md
  Open auto-memory folder
```

Lettura corretta

Cosa significa ogni riga e come interpretarla

1. **Auto-memory true**
   "Auto-memory true" = la memoria automatica è attiva.
2. **Opened ~/.claude/CLAUDE.md**
   Indica l'apertura del file delle istruzioni personali.
3. **Cancelled memory editing**
   Significa che è stata annullata un'operazione di editing/ selezione, non che la memoria sia stata cancellata.
4. **User instructions e Project instructions**
   Sono file di istruzioni, distinti dalla auto-memory.
5. **Open auto-memory folder**
   È la voce da usare per vedere i ricordi salvati automaticamente.

Che cosa significa ogni voce

Una guida rapida ai termini dell'output

Voce nell'output	Significato
Auto-memory	Funzione di memoria automatica (attiva o disattiva).
User instructions	Istruzioni personali globali, salvate in ~/.claude/CLAUDE.md.
Project instructions	Istruzioni del progetto corrente, salvate in ./CLAUDE.md.
Open auto-memory folder	Apre la cartella dei file di memoria salvati automaticamente.

Messaggio chiave

3 takeaway da ricordare

1. **Memoria attiva ≠ memoria già popolata.**
   Auto-memory attiva significa solo che la funzione è abilitata, non che ci siano già ricordi salvati.
2. **I file CLAUDE.md non sono la stessa cosa della auto-memory.**
   User instructions e Project instructions sono file di istruzioni, distinti dalla memoria automatica.
3. **Per verificare i ricordi effettivi, apri la cartella della auto-memory.**
   Usa "Open auto-memory folder" per vedere i file di memoria salvati.

Suggerimento hands-on: usa `/memory` per gestire istruzioni e auto-memory, e `/context` per capire quali file sono stati caricati nel contesto.

PICCOLI COMANDI,
GRANDI RISULTATI

Il comando `/memory` di Claude Code apre il pannello di gestione dei **file di memoria** della sessione: è il punto in cui si vede, si modifica e si configura tutto ciò che Claude Code "ricorda" tra una sessione e l'altra. Attenzione: è una memoria diversa da quella dell'app Claude di cui parlavamo prima — stesso concetto (persistenza tra sessioni), ma archivi separati: quella dell'app segue l'account, quella di Claude Code vive sul computer, legata ai progetti.

Nel dettaglio, `/memory` riporta:

1. L'elenco dei file CLAUDE.md — i file di istruzioni scritti *dall'utente*, su più livelli, dal più generale al più specifico:

- **User**: `~/.claude/CLAUDE.md` — vale per tutti i nostri progetti
- **Project**: `./CLAUDE.md` (o `./.claude/CLAUDE.md`) — condiviso col team via git
- **Local**: `./CLAUDE.local.md` — personale, da mettere nel `.gitignore`
- (esiste anche un livello **managed policy** aziendale, es. `/etc/claude-code/CLAUDE.md` su Linux, ma di norma non compare nell'elenco di `/memory`)

L'elenco mostra anche i file che non esistono ancora; selezionandone uno, viene creato e aperto nell'editor (VS Code si apre in una finestra separata, un editor da terminale come Vim prende il controllo del terminale).

2. Il controllo della auto memory — la memoria scritta *da Claude stesso*: appunti su preferenze, correzioni e contesto del progetto che Claude salva automaticamente in `~/.claude/projects/<progetto>/memory/`. Lì c'è un indice `MEMORY.md` (caricato a inizio sessione, fino a 200 righe o 25KB) più file tematici letti al bisogno. Da `/memory` si attiva/disattiva questa funzione (l'impostazione `autoMemoryEnabled` in `~/.claude/settings.json`) e si apre la cartella che la contiene. È attiva per default nelle sessioni locali.

La distinzione utile da ricordare: **CLAUDE.md è ciò che scriviamo noi, la auto memory è ciò che scrive Claude** — e `/memory` è il posto unico per gestire entrambe. Dire a Claude "ricordati che usiamo pnpm" finisce nella auto memory; dire "aggiungilo al CLAUDE.md" (o modificarlo via `/memory`) scrive nel file di istruzioni.

Nota pratica: per vedere quali file di memoria sono stati *effettivamente caricati* nel contesto della sessione corrente, il comando giusto è `/context`, sezione "Memory files" — il che ci riporta al discorso di prima: anche questi file consumano token della finestra di contesto.

Sources: Memory — code.claude.com, Slash commands — code.claude.com

Tabella riassuntiva

	Finestra di contesto	Gestione automatica del contesto	Memoria
Cos'è	Limite fisico del modello	Funzione dell'app (compressione)	Funzione dell'app (archivio)
Ambito	Una conversazione	Una conversazione	Tutte le conversazioni
Persistenza	Nessuna	Nessuna (fuori dalla chat)	Sì
Contenuto	Tutto, letterale	Riassunti del passato della chat	Fatti durevoli selezionati
Scopo	Vedere per rispondere	Non sbattere contro il limite	Continuità tra sessioni

In una frase: la finestra di contesto è *quanto* Claude può vedere ora; la gestione automatica decide *cosa tenere in vista* quando lo spazio finisce; la memoria decide *cosa portarsi dietro* da una conversazione all'altra. Se ti serve per un corso, i tre livelli si mappano bene su concetti classici: memoria di lavoro, compressione/summarization, e storage persistente con retrieval.

link alla docs di Claude Code sul prompt caching

CORE CONCEPTS

How Claude Code uses prompt caching

Claude Code manages prompt caching automatically. See why a model switch triggers a slow uncached turn, what `/compact` costs, why CLAUDE.md edits don't apply mid-session, and how to check your cache hit rate.

Prompt caching makes Claude Code faster and more cost-efficient. Without caching, the API would reprocess your full history on every turn. With caching, it reuses what it already processed, bills the re-read at the **cached token rate**, and fully processes only what changed.

Claude Code handles prompt caching for you, unless you **disable it**. It is still useful to know how prompt caching works, because some actions invalidate the cache and make the next response slower and more expensive while it rebuilds. This page covers which actions those are, why some settings wait for a restart to apply, and how to check cache performance when usage looks high.

/compact E CONTEXT WINDOW (CW)

Si possono perdere informazioni importanti?

SÌ: LA COMPATTAZIONE È LOSSY.

Nella CW, la conversazione integrale viene sostituita da un riassunto che può perdere dettagli.

COSA RESTA, COSA SI RIDUCE



Nel riassunto

Richieste e intento; concetti e decisioni chiave; file e snippet rilevanti; errori risolti; task aperti e lavoro in corso.



Dettagli a rischio

Output integrali dei tool, passaggi intermedi e codice letto parola per parola. Il codice esatto può richiedere una nuova lettura.

RICARICAMENTI MENO OVVI

CLAUDE.md di progetto e regole generali: reiniettati.

Regole con paths: e CLAUDE.md annidati: tornano quando si legge, scrive o modifica un file pertinente.

Skill: nella simulazione ufficiale, l'elenco non viene reiniettato; i corpi delle skill già invocate tornano entro limiti.

QUATTRO MOSSE PER LIMITARE LE PERDITE

1 · DARE UN FOCUS

```
/compact focus on
bug di login, decisioni e test
```

Guida il riassunto; non garantisce zero perdite.

2 · SCEGLIERE IL MOMENTO

Preferire fine task o un confine pulito. Nel debug aperto, fissare prima ipotesi, prove e risultati.

3 · SALVARE E RILEGGERE

```
docs/plan.md: stato, decisioni
e prossimi passi. Chiederne la
rilettura dopo /compact.
```

Regole stabili nel CLAUDE.md di progetto.

4 · CAMBIARE CONVERSAZIONE

```
/clear
```

Per un nuovo lavoro indipendente: ripartire con una conversazione pulita.

SALVARE LO STATO SU FILE E RILEGGERLO DOPO LA COMPATTAZIONE.

Fonti: Claude Code Docs · Context window · Memory · How Claude Code works

Claude Code: configurazioni essenziali

I file e le regole da conoscere per iniziare

ISTRUZIONI · Markdown
Testo che Claude legge come guida

IMPOSTAZIONI · JSON
Regole e opzioni applicate dal programma

4 FILE DA CONOSCERE

`CLAUDE.md`
Progetto · istruzioni condivise
Build, test e convenzioni. Breve e chiaro. In Git.

`~/.claude/settings.json`
Utente · tutti i progetti
Preferenze, modello e permessi personali.

`.claude/settings.json`
Progetto · team
Impostazioni comuni da condividere con Git.

`.claude/settings.local.json`
Progetto · personali
Override e approvazioni persistenti. Da escludere da Git.

~ = cartella home. Su Windows: %USERPROFILE%/.claude

PERMESSI · permissions

`allow` = consenti `ask` = chiedi conferma `deny` = blocca

Tra regole applicabili: `deny` > `ask` > `allow`

Esempio in `~/.claude/settings.json`

```
{
   "permissions": {
     "allow": ["Bash(npm test *)"],
     "deny": ["Read(./.env)", "Read(./.env.*)"]
   }
}
```

Test senza conferma; lettura dei file .env limitata.

Da sole, le regole Read non bloccano ogni lettura indiretta tramite script.

COMANDI UTILI

`/config`	`/status`	`/memory`	`claude doctor`
Opzioni personali	Fonti settings caricate	CLAUDE.md e auto-memory	Diagnostica dal terminale

PRECEDENZA DELLE IMPOSTAZIONI

Dalla più alta alla più bassa

Azienda → **CLI (`--settings`)** → **Locale** → **Condiviso** → **Utente**

I valori singoli si sovrascrivono. Le liste di permessi si uniscono.

VS Code: autocompletamento e validazione con $schema

```
"$schema": "https://json.schemastore.org/claude-code-settings.json"
```

I file settings possono mancare all'inizio: vengono creati quando servono.

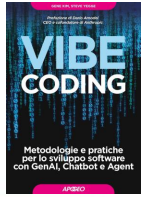
ATTENZIONE

- JSON: niente commenti o virgole finali.
- ~/.claude.json: stato interno dell'app.
- .claudeignore non è supportato.
- Non stampare segreti: le sessioni si salvano in chiaro.

Fonti: documentazione ufficiale Claude Code



Il vibe coding



Evoluzione del coding con l'AI

Dalla programmazione tradizionale al vibe coding

1 Programmazione manuale o tradizionale

pre-AI



Il codice viene scritto a mano dal programmatore.

Ruolo umano
Scrive tutto il codice.

Ruolo dell'AI
Assente.

2 Completamento automatico del codice

dal 2021



L'IDE suggerisce o **completa** porzioni di codice basandosi su ciò che è stato digitato.

Ruolo umano
Scrive il codice e accetta o modifica i suggerimenti.

Ruolo dell'AI
Suggerisce completamenti locali.

3 Programmazione a chat
chat coding

dal 2023



Si chiede all'AI di generare o modificare codice in una conversazione turno per turno.

Ruolo umano
Formula richieste e integra le risposte.

Ruolo dell'AI
Genera codice e propone modifiche.

4 Programmazione agentica
agentic coding

inizio 2025



L'agente AI modifica file, usa strumenti, esegue test e lavora in modo più autonomo.

Ruolo umano
Assegna obiettivi e supervisiona.

Ruolo dell'AI
Implementa, testa e perfeziona.

5 Vibe coding

oggi / emergente



Il programmatore delega molto della scrittura del codice all'AI, ma resta attivo: supervisiona, valuta e critica i risultati.

Ruolo umano
Definisce obiettivi, supervisiona e valida.

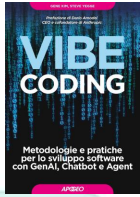
Ruolo dell'AI
Scrive gran parte del codice e propone soluzioni.

Importante: nel testo il vibe coding non è presentato come una nuova tecnologia separata, ma come un modo di lavorare con gli strumenti AI, soprattutto quelli agentici.

Come cresce la delega all'AI



In sintesi: l'evoluzione non elimina il programmatore; cambia il suo ruolo, da autore diretto del codice a supervisore sempre più strategico.



Origine del termine ‘vibe coding’

Sintesi basata solo sulle informazioni emerse in questa chat

1 Che cosa sappiamo sull’origine

> Il termine viene attribuito ad Andrej Karpathy per descrivere un nuovo modo di programmare.

2 Chi viene indicato come autore del termine

Andrej Karpathy

È indicato come la persona che ha coniato l’espressione ‘vibe coding’.

L’attribuzione del termine viene riportata nelle informazioni considerate qui.

3 Che cosa significa ‘vibe coding’

- L’AI scrive il codice per noi.
- Non si digita più il codice a mano come attività centrale.
- L’AI aiuta lungo tutto il ciclo di vita del software: architettura, soluzioni, implementazione, test, sicurezza.
- La persona dirige, verifica e mantiene la visione; l’AI si fa carico dell’implementazione.

In sintesi: è vibe coding quando si dirige più di quanto si digiti.

4 Che cosa NON sappiamo da queste informazioni

- Non è indicato quando il termine è stato coniato.
- Non è indicato dove è stato introdotto.
- Non è spiegato perché sia stato scelto il nome ‘vibe coding’.
- Non sono disponibili qui i dettagli storici completi del conio.

Importante: questa infografica riporta solo le informazioni emerse nella chat, senza aggiunte esterne.

Messaggio chiave: il termine è attribuito a Andrej Karpathy, ma qui non sono disponibili i dettagli storici completi del conio.



I 5 benefici del vibe coding: FAAFO

Il vibe coding ti permette di realizzare "cose" più velocemente, di essere più ambizioso su ciò che puoi creare, di farlo con maggiore autonomia, di divertirti di più e di esplorare più opzioni. Questi cinque aspetti sono riassunti nell'acronimo FAAFO.



F — Faster

Scrivi codice più velocemente.



- Compiti che prima richiedevano mesi o settimane, ora possono essere svolti in un giorno.
- Compiti che richiedevano giorni, ora possono essere completati in ore.
- L'accelerazione deriva non solo dalla generazione di codice, ma anche dall'aiuto dell'AI con debugging, test e documentazione.
- Progetti rimasti in sospeso per anni possono finalmente vedere la luce.

A — Ambitious

Puoi essere più ambizioso su ciò che puoi realizzare.



- Porta alla tua portata progetti apparentemente impossibili.
- Rende più facili da affrontare anche piccoli compiti con ROI marginale.
- Questo è dovuto alla velocità, alla vasta conoscenza e alle capacità dell'AI.
- Rimodella il tuo approccio allo sviluppo, eliminando molti dei dolorosi compromessi che da sempre limitano ciò che viene realizzato.

A — Autonomously

Puoi lavorare con maggiore autonomia.



- Puoi completare cose che prima richiedevano più persone o team.
- Funzionalità che un tempo richiedevano specialisti di più discipline possono essere gestite da un singolo sviluppatore non specializzato.
- Elimina due costose "tasse": riduce i costi di coordinamento (riunioni, allineare le priorità, aspettare la disponibilità) e le sfide comunicative.
- Lavorare in modo più autonomo con l'AI riduce o rimuove significativamente questi ostacoli.

F — Fun

La programmazione diventa più divertente.



- Ti risparmia le parti meno piacevoli della programmazione, come il debugging degli errori di sintassi, la lotta con librerie sconosciute o il cambio dell'infrastruttura di test.
- Puoi concentrarti sulla risoluzione dei problemi degli utenti, sulla realizzazione di cose interessanti e sul portare a termine i compiti.
- Lavorare con l'AI è anche stranamente avvincente.
- Sta allontanando le persone dal pensionamento, sta attirando non programmatori e sta incoraggiando i leader ad assumersi una quota maggiore del lavoro di programmazione.

O — Options

Puoi esplorare più opzioni.



- Aumenta la tua capacità di esplorare opzioni, sia per trovare una soluzione sia per mitigare i rischi.
- Invece di impegnarti in un unico approccio, puoi prototipare rapidamente molteplici modi per risolvere il problema.
- Puoi valutare i loro compromessi.
- Quando riconosci un problema in cui l'esplorazione aiuterebbe, puoi attivare di riflesso indagini parallele.



FAAFO: un modo di lavorare più veloce, più ambizioso, più autonomo, più divertente e con più opzioni.

Vibe coding per persone adulte



Mentre gli influencer di YouTube catturano i titoli generando simulatori di volo della Seconda guerra mondiale con un singolo prompt, noi ci concentriamo sul portare il vibe coding nell'ingegneria del software professionale. Questo richiede l'applicazione di pratiche ingegneristiche disciplinate, lasciando comunque all'AI il compito di gestire i noiosi dettagli implementativi. In altre parole, **è vibe coding per persone adulte.**

1 Che cosa significa

Fare tutte quelle cose da persone adulte di cui potreste già essere responsabili.



- Revisioni della sicurezza.
- Copertura dei test.
- Gestione del raggio d'esplosione.
- Eccellenza operativa.

2 Velocità e risultati

Lo fate a una velocità che nessuno di noi ha mai sperimentato prima, creando migliaia (potenzialmente anche decine di migliaia) di righe di codice al giorno.



3 Applicazioni reali

Quando lavorate all'autenticazione per un'applicazione rivolta ai clienti, esaminerete ancora ogni riga di codice e realizzerete suite di test complete – ma potete farlo molto più velocemente.



4 Sistemi legacy

Per i sistemi legacy che nessuno ormai comprende più, potreste usare l'AI per analizzare e documentare la base di codice e poi per realizzare test per catturare il comportamento esistente; solo allora potrete iniziare a fare modifiche con una certa fiducia.



5 Esempio concreto

Ecco un ottimo esempio di modifica di codice legacy: il ricercatore Microsoft Jonathan Larson ha dimostrato l'uso di LLM e GraphRAG per modificare il codice sorgente di DOOM di id Software (anno 1993), abilitando la possibilità per il giocatore di saltare. È stata un'impresa non banale, perché. il motore originale non ha un vero modello 3D interno ed era costruito sul presupposto che il giocatore fosse sempre poggiato a terra. La modifica ha coinvolto molti sottosistemi fortemente accoppiati, tra cui la fisica, lo stato del giocatore, la gestione degli input e la logica di livello.[13]



6 Il vostro ruolo

Si tratta di prendere la vostra disciplina ingegneristica così duramente guadagnata e applicarla con maggiore intensità. Voi siete l'head chef, e il vostro ruolo è quello di stabilire gli standard, assaggiare rigorosamente e assicurare che ogni piatto soddisfi i vostri standard, perché, se la cucina accelera, aumenta notevolmente anche la potenziale frequenza e dimensione degli errori.



7 Il momento è adesso

Come sottolinea Karpathy, questi strumenti AI stanno migliorando esponenzialmente. Quelli di oggi sono i meno avanzati che vedrete. Con questo in mente, crediamo sia giunto il momento di andare oltre la scrittura meticolosa di ogni riga di codice per adottare pienamente questo nuovo approccio allo sviluppo software.



Ecco qualcosa in cui crediamo sinceramente: nessuno dovrebbe più scrivere codice a mano se non è necessario.

Quali competenze imparare



Il mondo sta cercando di capire che cosa cambia e non cambia quando uno sviluppatore usa l'AI su tutto ciò su cui sta lavorando, e quali competenze sono le più importanti in questo nuovo mondo. Poiché gli strumenti si evolveranno rapidamente, **i principi di fondo dell'ingegneria del software tradizionale** giocheranno un ruolo almeno altrettanto grande, se non maggiore, di prima. Pertanto, è essenziale:

1 Creare cicli di feedback veloci e frequenti



Creare cicli di feedback veloci e frequenti per la **validazione** e il **controllo.**

2 Curare la modularità



Curare la modularità per **ridurre la complessità**, abilitare il **lavoro parallelo** ed **esplorare opzioni.**

3 Accogliere l'apprendimento



Accogliere l'apprendimento in un mondo in cui **tutto cambia velocemente.**

4 Padroneggiare la professione



Padroneggiare la professione, per prosperare in un ambiente in cui in un breve lasso di tempo tutto il lavoro della conoscenza **cambierà in modo radicale.**



Impairare queste tecniche sarà critico **per tutti coloro che operano nel campo della conoscenza**, non solo per gli sviluppatori e i vibe coder.



Il ciclo del vibe coding

Le fasi del ciclo e il passaggio bonus di automazione

Prima di entrare nell'esempio di Gene sulla scrittura di un generatore di estratti video, parliamo del *ciclo del vibe coding*. Ecco le sue fasi.



1 Inquadrare l'obiettivo

Fornite al vostro assistente AI una panoramica chiara e concisa del risultato che state cercando. Siate specifici sull'aspetto del successo e sul perché lo state costruendo.

2 Scomporre i compiti

Suddividete ciò che state cercando di fare in passaggi chiari e realizzabili. In generale, più piccoli sono i passaggi, maggiori sono le possibilità che l'AI abbia successo. Anche man mano che l'AI diventa più abile, è sempre una buona idea specificare passaggi piccoli. Non esitate a chiederle di suddividere i grandi compiti (per esempio: 'Ecco cosa sto cercando di fare. Proponimi un piano.').

3 Iniziare la conversazione

Chiedete all'AI di generare un piano per raggiungere il vostro obiettivo, o datele istruzioni per iniziare.

4 Revisionare con attenzione

La soluzione che la vostra AI proporrà potrebbe sembrare corretta, ma fino a quando non avrete stabilito una base di fiducia, dovete ricontrollarla.

5 Collaudare e verificare

Siete sempre voi i responsabili della qualità del codice, anche se l'ha scritto l'AI. Tutto funziona meglio se scrivete i vostri test e le vostre aspettative prima di generare il codice, come raccomandano i sostenitori dello sviluppo guidato dai test (TDD). Fallite velocemente, correggete velocemente e chiedete all'AI di aiutarvi a individuare quegli errori subdoli che potrebbero rimanere inosservati.

6 Perfezionare e iterare

Continuate a iterare finché non raggiungerete il vostro obiettivo. Questo ciclo del vibe coding somiglia al ciclo di sviluppo tradizionale. Ma quando si programma con l'AI, ogni passaggio diventa critico. Non potete addormentarvi al volante, o presto vi ritroverete con un frustrante e costoso rifacimento.

BONUS

7 Automatizzare il flusso di lavoro

Iniziate ad automatizzare parti del vostro flusso di lavoro. Qualsiasi attrito crea enormi costi-opportunità. Tutto il tempo che passate a digitare o copiare/incollare/spostare rallenta il ciclo di vibe coding. Se state facendo qualcosa manualmente, ricordatevi che questo è un costo, che pagate ogni volta che provate a fare vibe coding.

Automatizzare questa fatica non solo vi renderà più veloci, ma accelererà la vostra capacità di sperimentare e innovare.

Se, in qualsiasi momento, vi trovate a digitare molto o a cercare manualmente in strutture dati, fermatevi e chiedetevi: 'Potrei chiedere all'AI di aiutarmi?' La risposta di solito è sì, e così sarete più veloci e vi divertirete di più.

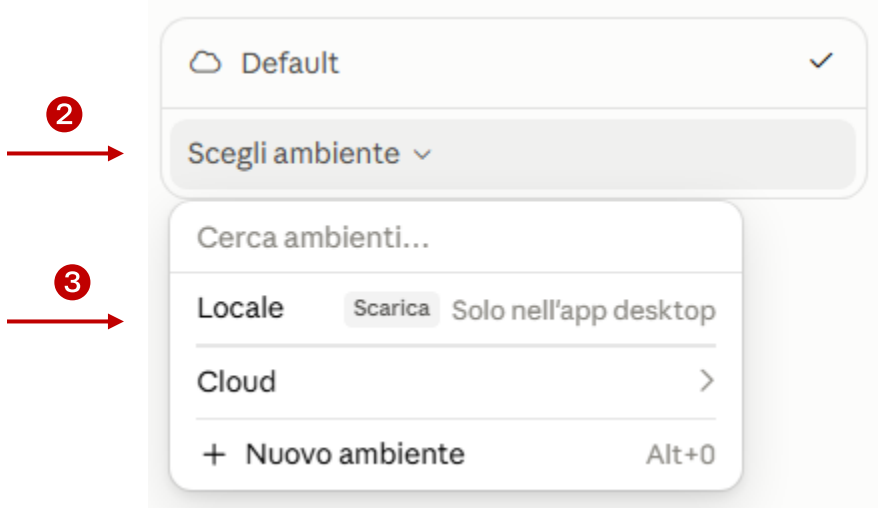
NOTA — Consigliamo vivamente di utilizzare un gestore di Appunti, se già non lo state facendo. Con il vibe coding ci si trova spesso a passare continuamente da uno strumento all'altro. Tra i gestori di Appunti più noti troviamo Alfred o Paste per macOS, Ditto o ClipClip per Windows, CopyQ per Linux (ma è multi-piattaforma) o la cronologia degli Appunti integrata in Windows (Win+V). Attualmente i gestori di Appunti offrono notevoli miglioramenti al flusso di lavoro che impiega l'ingegneria dei prompt e la gestione del contesto.

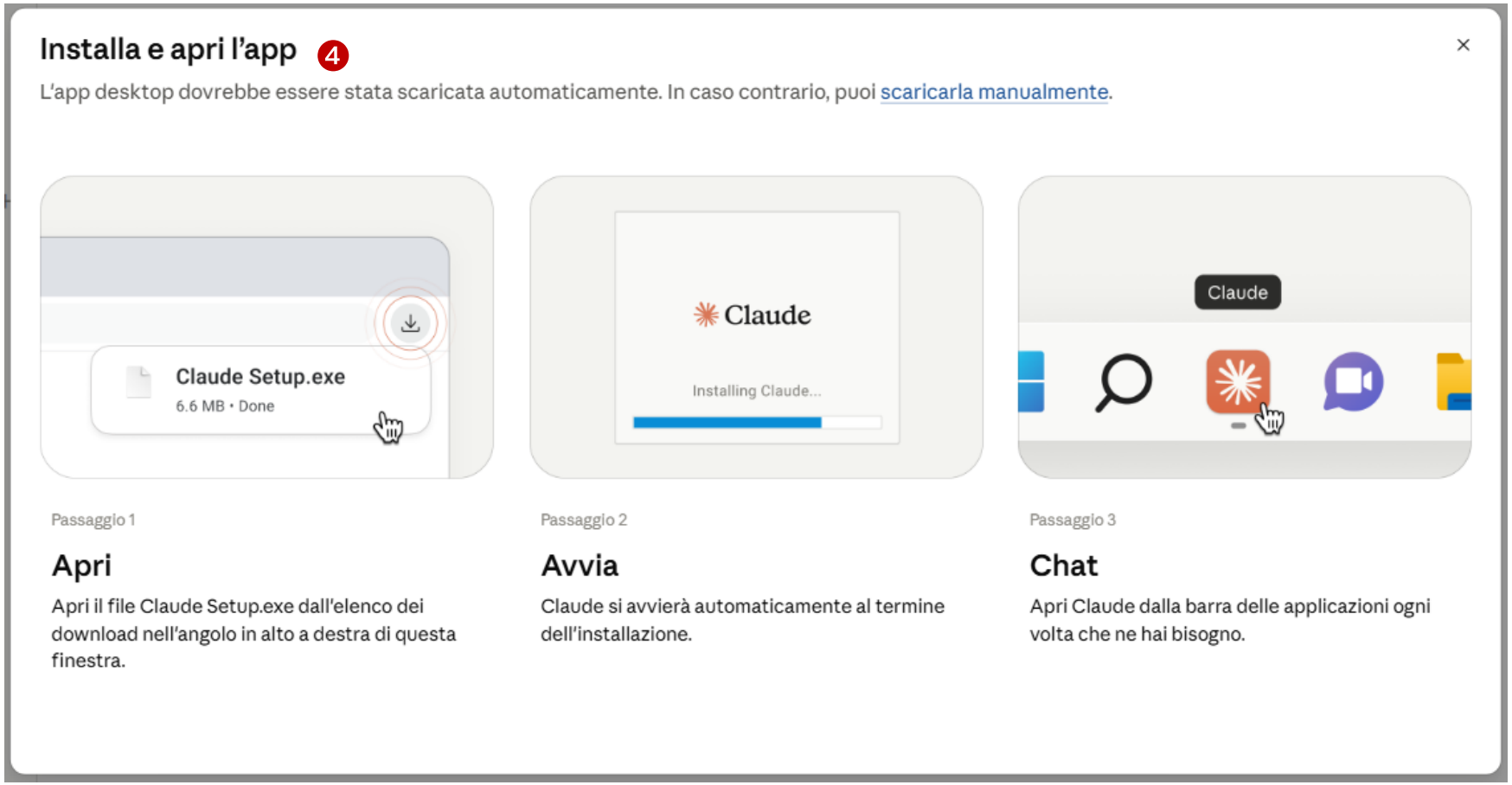


Corso: slide operative

Installazione di Claude Code







PowerShell (nativo Windows)

Pro

Già installato, zero setup
Pipeline di oggetti, non testo
Percorsi Windows senza problemi
JSON gestito nativamente

Contro

Più verboso negli one-liner
Meno esempi e tutorial online
Non gira sui server Linux
Execution policy da gestire

Bash (Git Bash / WSL)

Pro

Standard del mondo tecnico
Conciso, pipe potenti
Vale su Mac, Linux, server
WSL: Linux vero, GPU CUDA

Contro

Va installato a parte
Percorsi Windows ostici
Git Bash: emulazione parziale
WSL: filesystem separato

Quale scegliere

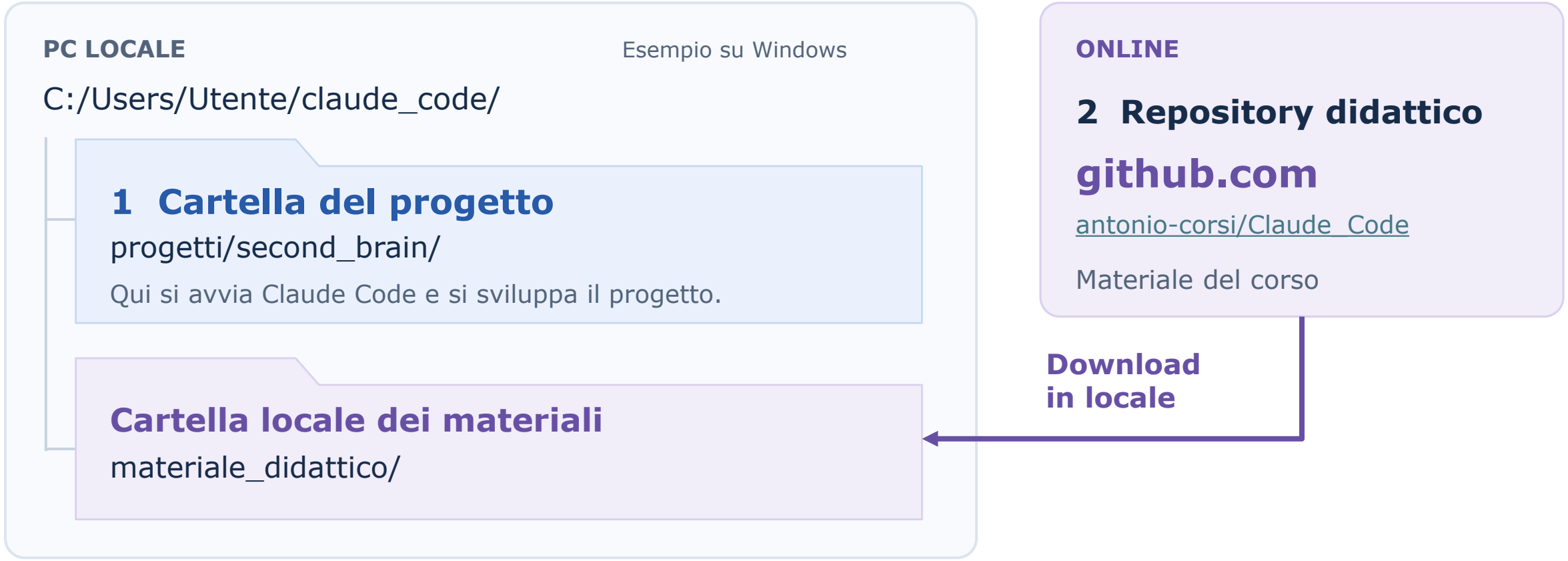
PowerShell
Script locali Windows

Git Bash
Comandi Unix leggeri, Git

WSL
Linux vero, tutorial, CUDA

I due ambienti del corso

Il progetto si sviluppa in locale. I materiali del corso arrivano da GitHub.



La cartella dei materiali deve restare **fuori dalla cartella del progetto**.

Questo PC › OS (C:) › Utenti › Utente › claude_code › progetti

github.com/antonio-corsi/Claude_Code

antonio-corsi / Claude_Code

Type / to search

Code | Issues | Pull requests | Agents | Actions | Projects | Wiki | Security and quality | Insights | Settings

Claude_Code

Public

Pin | Watch 0 | Fork 0 | Star 0

main | 1 Branch | 0 Tags | Go to file T | Add file | Code

antonio-corsi Materiale iniziale — c7ca275 · 20 minutes ago — 2 Commits

Intro di Claude AI	Materiale iniziale	20 minutes ago
demo weather-globe	Materiale iniziale	20 minutes ago
doc-in-md	Materiale iniziale	20 minutes ago
.gitattributes	Initial commit	21 minutes ago
CC dove rende di più.png	Materiale iniziale	20 minutes ago
Cover faldone.docx	Materiale iniziale	20 minutes ago
Gian Segato statement on CC.png	Materiale iniziale	20 minutes ago
In Claude Code CLI sono disponibili 6 permissi...	Materiale iniziale	20 minutes ago
Installazioni_preliminari.docx	Materiale iniziale	20 minutes ago
Installazioni_preliminari.pdf	Materiale iniziale	20 minutes ago
Intro Claude Code.pptx	Materiale iniziale	20 minutes ago
Modello di Slide Infografica Techқap.png	Materiale iniziale	20 minutes ago
README.md	Initial commit	21 minutes ago
combocoloriTerminale.json	Materiale iniziale	20 minutes ago
finestra_contesto.png	Materiale iniziale	20 minutes ago

About

Repository del materiale didattico di Overnet per il corso "Claude Code: sviluppo software con AI agentica" a Poste Italiane (ottobre 2026)

Readme
Activity
0 stars
0 watching
0 forks

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Contributors 1

antonio-corsi

COSA FA statusline_prompt.md

Una specifica per creare una status line minimale e informativa in Claude Code

In breve
Il file descrive il risultato da generare; non contiene ancora lo script Bash finale.

DAL FILE ALLA STATUS LINE



RISULTATO RICHIESTO

```
main+* · Modello (high) · 67%
```

Tre segmenti, separati da un punto centrale “·” in grigio attenuato.

1. BRANCH GIT

Mostra il nome del branch corrente in ciano. Il segmento compare solo quando ci si trova in un repository Git.

`main`	branch pulito
`main*`	* arancione = modifiche non in stage
`main+`	+ giallo = modifiche in stage

Fuori da Git: il segmento non appare

2. MODELLO ED EFFORT

Mostra il nome visualizzato del modello in blu, seguito dal livello di effort tra parentesi in grigio attenuato.

```
Modello (high)
```

Origine dell’effort:

`~/.claude/settings.json`

Campo letto `effortLevel`

3. UTILIZZO DEL CONTESTO

Mostra soltanto il numero e il simbolo %. Il colore cambia in base alla percentuale di contesto utilizzata.

32%	normale: verde
58%	dal 50%: giallo

Nessuna etichetta “ctx”

ASPETTO VISIVO

- Stile minimale
- Separatori “·” in grigio dim
- Colori ANSI discreti
- Nessuno sfondo colorato

COSA NON MOSTRA

- ✕ nome utente o host
- ✕ directory corrente
- ✕ etichetta “ctx”
- ✕ informazioni non richieste

IN PRATICA

Il file è una specifica operativa: dice a Claude Code come configurare una status line tramite uno script Bash.

Obiettivo: informazioni essenziali a colpo d’occhio

Fonte: statusline_prompt.md allegato

Formato 16:9 · sintesi fedele

non è idempotente, se si vuole rieseguirlo cambiare cartella

DAL PROMPT AL PROGETTO PYTHON

rispetto all'originale, inserita una clausola di salvaguardia dopo le tre domande (❶)

Che cosa chiede a Claude Code il prompt di scaffolding

letter. «impalcatura, ponteggio»: nella IT «struttura di base»

1 • CHIEDE

Navigazione con le frecce
- left-right → sceglie il tab
- up-down → sceglie il valore

Nome del progetto

second_brain

Non mettere MAI spazi nei nomi di cartelle e variabili dati a Claude Code; usare *underscore* «_»

Versione iniziale: 0.0.1

0.0.1

Cartella di destinazione

./progetti/second_brain

Non mettere MAI spazi nei nomi di cartelle e variabili dati a Claude Code; usare *underscore* «_»

Adatta i nomi per Python

2 • CREA E CONFIGURA

STRUTTURA

src/ • tests/ • docs/ • scripts/

Package installabile e avvio da terminale

AMBIENTE

Python ≥ 3.13 • uv • uv_build

pyproject.toml: dipendenze e versione

LOGGING

Loguru: console + file

Configurazione con variabili d'ambiente

TEST

pytest: verifica il saluto

Log dei test in una cartella temporanea

QUALITÀ

Ruff: controllo e formattazione

Copertura: soglia configurata 80%

DOCUMENTAZIONE

MkDocs + Material • README

Guida d'uso e riferimento API

3 • INSTALLA E VERIFICA

Windows PowerShell

```
PS C:\progetti\second_brain> uv sync
```

Installa progetto e dipendenze

Windows PowerShell

```
PS C:\progetti\second_brain> uv run pytest
```

Esegue il test iniziale

DA AVVIARE SEPARATAMENTE

Ruff • copertura • anteprima e build della documentazione

APP INIZIALE

Configura il logging e scrive un saluto.

Windows PowerShell

```
PS C:\progetti\second_brain> uv run second_brain
2025-03-08 10:15:27 | INFO | second_brain | Hello from second_brain!
```

RISULTATO

Una base organizzata per sviluppare le funzionalità dell'applicazione.

uv.lock da versionare • .env e .env.test esclusi da Git

I contenuti del prompt di scaffolding - 1

Due versioni del prompt di scaffoldding (nella cartella Claude Code › doc-in-md)

python_project_scaffold.md ← *la versione originale, da dare in input a Claude Code (**in plan mode**)*

python_project_scaffold_render.md ← *la versione modificata, **ben renderizzabile in VS Code***

La parte centrale del prompt (dopo le domande):

```
Requirements:

- **Package manager:** uv
- **Config:** pyproject.toml only (no setup.py, no setup.
- **Layout:** src/ layout, installable package
- **Type hints:** none
- **Versioning:** single source of truth in pyproject.tom
```

personalizzabili

→ *il package manager*

*Il file pyproject.toml **semplifica la configurazione dei progetti Python**, unificando la configurazione dei pacchetti, la gestione delle dipendenze e l'ottimizzazione dei processi di build.*

→ *il layout (sarà più chiaro più avanti)*

Che cosa sono i type hints?
*Sono annotazioni che specificano il **tipo di dato atteso** per i parametri di una funzione, per il risultato restituito o per una variabile. Aiutano a comprendere il codice e possono essere utilizzate dagli editor e dagli strumenti di analisi statica per segnalare possibili errori.*
Generano un pò di confusione visuale??
Valori alternativi: optional, public API only, all function signatures, full, ecc)

Seguono i contenuti di certi file.

I contenuti del prompt di scaffolding - 2

Note sul prompt:

- Doppia descrizione delle istruzioni: **umane** e **codificate**
  → migliore comprensione sia per Claude Code che per il manutentore umano di questo prompt (documentazione
- Il prompt evolverà (cambiano framework e specifiche, ecc)
- Diversi ambienti (sviluppo, test,ecc)
- Readme e standard .gitignore
- After scaffolding

```
### src/<packagename>/app.py
- Define a `configure_logging()` function that:
  1. Imports `os`
  2. Reads `LOG_LEVEL` from `os.environ.get("LOG_LEVEL",
  3. Reads `LOG_FILE` from `os.environ.get("LOG_FILE", "a
  4. Removes the default loguru handler
  5. Adds a stderr handler at the `LOG_LEVEL` level (defa
  6. Adds a file handler at DEBUG level using the `LOG_FI
```

```
```python
def configure_logging():
    """Configure loguru for console and file logging.

    Removes the default handler and sets up:
    - stderr handler at LOG_LEVEL (default: INFO, confi
    - File handler at DEBUG level writing to LOG_FILE (
    """
    import os
    log_level = os.environ.get("LOG_LEVEL", "INFO")
    log_file = os.environ.get("LOG_FILE", "app.log")
    logger.remove()
    logger.add(sys.stderr, level=log_level)
    logger.add(log_file, level="DEBUG", rotation="50 KB
```
```

After scaffolding: 1. Run `uv sync` to set up the environment 2. Run `uv run pytest` to verify the starter test passes 3. Remind the user to: - Commit `uv.lock` to version control for reproducible installs - Try `uv run python scripts/serve_docs.py` to preview the documentation locally ```

Il piano

Plan: Scaffold second-brain Python Project

Context

Scaffold a best-practice Python application in the current directory /Users/rp/projects/cclc/sb.
- **Project name** (pyproject.toml, docs): second-brain
- **Package name** (Python imports/dirs): second_brain
- **Version:** 0.0.1
- **Location:** current directory (scaffolded directly into /Users/rp/projects/cclc/sb)

File Structure to Create

```
/Users/rp/projects/cclc/sb/
  src/
    second_brain/
      __init__.py          (empty)
      __main__.py          (calls main() from app.py)
      app.py               (configure_logging + main)
  tests/
    __init__.py            (empty)
    conftest.py            (autouse log-file fixture)
    test_app.py            (test_main_logs_greeting)
  scripts/
    serve_docs.py          (mkdocs serve + tee to
```

Key Decisions

Names

- Package dir: src/second_brain/
- CLI entrypoint: second_brain = "second_brain.app:main"
- Docs reference :::second_brain

pyproject.toml highlights

- Build backend: uv_build>=0.10.9,<0.11.0
- requires-python = ">=3.13"
- Dependencies: loguru
- Dev group: ruff, pytest, pytest-cov, pytest-env, mkdocs, mkdocs-material, mkdocstrings[python]
- Ruff: line-length=88, target-version="py313", select=["E","F","B","I","UP"], ignore=["E501"]
- pytest: --import-mode=importlib, --strict-markers, --strict-config, -ra, testpaths=["tests"]
- pytest-env: loads .env.test
- Coverage: branch=true, source=["src"], fail_under=80

app.py

- configure_logging(): reads LOG_LEVEL/LOG_FILE from env, removes default handler, adds stderr + rotating file handlers
- main(): decorated with @logger.catch, calls

Post-Scaffold Steps

1. Run uv sync to install dependencies and generate uv.lock
2. Run uv run pytest to verify the starter test passes
3. Remind user to:
   - Commit uv.lock for reproducible installs
   - Try uv run python scripts/serve_docs.py to preview docs

Verification

- uv run pytest → 1 test passes (test_main_logs_greeting)
- uv run second_brain → prints "Hello from second_brain!" to stderr + writes app.log
- uv run pytest --cov → coverage ≥ 80%

Claude has written up a plan and is ready to execute. Would you like to proceed?

❯ 1. Yes, auto-accept edits
  2. Yes, manually approve edits
  3. Type here to tell Claude what to change

ctrl-g to ediVS Cod · ~/.claude/plans/hazy-weaving-u
in mbrella.md

Come rieseguire il prompt di scaffolding - 1

Procedura consigliata per il corso

1. **Creare una nuova cartella vuota**, esterna al progetto della sessione precedente.
2. **Aprire un terminale nella nuova cartella e avviare una nuova sessione di Claude Code** con `claude`. Avviare Claude dalla directory del progetto è il flusso documentato da Anthropic.
3. **Fornire il prompt completo.**
4. Alla terza domanda del prompt, **scegliere "current directory"**: il progetto verrà creato direttamente nella cartella appena predisposta.

Il file del prompt può rimanere nella cartella dei materiali del corso, così la nuova destinazione resta vuota.

Se viene scelta invece l'opzione "**new subdirectory**", la root effettiva sarà la sottocartella creata da Claude: **eseguire anche uv sync, i test e gli altri comandi da quella posizione**. Questa distinzione è già prevista nelle istruzioni del file.

Che cosa significa, esattamente, "ripartire da zero"

Una nuova root permette di **ricreare da zero i file del progetto**. Alcuni elementi della macchina rimangono però disponibili:

- **La cache di uv**, che può riutilizzare pacchetti già scaricati e rendere la seconda esecuzione più rapida.
- **Le installazioni Python già presenti**. uv può riutilizzare un interprete compatibile e, quando necessario, scaricarne uno.
- **Le impostazioni utente di Claude Code**, che si applicano anche ai nuovi progetti.

Per ripetere l'esercitazione di scaffolding, questo riuso è normalmente appropriato. La cartella autonoma presuppone inoltre che il nuovo progetto non venga inserito in un *workspace uv* già configurato per gestire più package insieme.

Come rieseguire il prompt di scaffolding - 2

Nuova esecuzione e riproducibilità non sono la stessa cosa

Cambiare root non rende il prompt idempotente: consente di eseguirlo su una nuova destinazione. Il prompt attuale non specifica una politica completa per gestire file già esistenti, quindi non andrebbe trattato come un'operazione sicura da ripetere indiscriminatamente nella stessa cartella.

Inoltre, per ottenere **le stesse identiche versioni delle dipendenze in lezioni successive**, una nuova root non basta: nel prompt ci sono requisiti aperti, come `ruff>=0.11` e `pytest>=8.0`. Occorre conservare una versione di riferimento del progetto con `pyproject.toml`, `uv.lock` e una versione Python fissata. Il lockfile registra le versioni risolte delle dipendenze.

Per l'uso durante il corso si consiglia di aggiungere al prompt un controllo iniziale della destinazione: mostrare il percorso assoluto, verificare che la cartella sia vuota — inclusi i file nascosti — e fermarsi se contiene già qualcosa. Questa regola renderebbe esplicito il comportamento atteso prima di creare file o avviare `uv sync`.

uv — Il tool moderno per lo sviluppo Python

Un unico strumento, tutto quello che ti serve

Cos'è uv?

uv è un **programma da riga di comando (CLI)** open source di Astral, scritto in **Rust**, che rende più semplice, veloce e affidabile lo sviluppo Python.
Gestisce versioni di Python, ambienti virtuali, dipendenze, esecuzione di strumenti e progetti, con un'unica interfaccia.

- **Veloce** — Scritto in Rust, estremamente rapido
- **Affidabile** — Risoluzione delle dipendenze consistente
- **Completo** — Ambiente, dipendenze, Python e strumenti
- **Moderno** — Progettato per i workflow attuali
- **Open source** — Sviluppato da Astral

Cosa sostituisce?

uv riunisce le funzionalità di diversi strumenti tradizionali in un'unica soluzione.

Strumento tradizionale		Funzionalità equivalente in uv
pip	→	Installazione dei package Python
venv	→	Creazione di ambienti virtuali
pip-tools	→	Risoluzione e blocco delle dipendenze
pyenv	→	Installazione e gestione delle versioni Python
pipx	→	Esecuzione di strumenti Python isolati
poetry	→	Gestione di progetti e dipendenze

Cosa puoi fare con uv?

I comandi principali per le operazioni più comuni.

Gestire Python
Installare e gestire diverse versioni di Python.

```
uv python install 3.13
uv python list
```

Creare un progetto
Inizializzare un nuovo progetto Python.

```
uv init myproject
cd myproject
```

Gestire dipendenze
Aggiungere, rimuovere e sincronizzare i package.

```
uv add pandas
uv remove pandas
uv sync
```

Eseguire comandi
Lanciare script o strumenti nell'ambiente del progetto.

```
uv run python main.py
uv run pytest
uv run ruff check .
```

Strumenti isolati
Eseguire strumenti senza installarli globalmente.

```
uv tool run ruff
uv tool install mkdocs
```

Esempio pratico: ciclo di vita di un progetto

Dalla creazione all'esecuzione, in pochi comandi.

1	**Crea il progetto**	`uv init analisi-clienti`
2	**Entra nella cartella**	`cd analisi-clienti`
3	**Aggiungi dipendenze**	`uv add loguru pandas`
4	**Aggiungi dipendenze di sviluppo**	`uv add --group dev pytest ruff`
5	**Prepara l'ambiente**	`uv sync`
6	**Esegui l'applicazione**	`uv run python main.py`
7	**Esegui i test**	`uv run pytest`
8	**Esegui strumenti**	`uv run ruff check .` `uv run mkdocs serve`

Come si installa?

Puoi installare uv in diversi modi.

Installer ufficiale (consigliato)

```
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Installa l'eseguibile e lo aggiunge al PATH %USERPROFILE%\.local\bin).

Tramite pip

```
pip install uv
```

Installa l'eseguibile nella cartella Scripts di Python.

Tramite Scoop (Windows) `scoop install uv`
Usa il gestore di package Scoop.

Dove si trova su Windows?

uv è un eseguibile da riga di comando, non un'app grafica. Può non comparire nell'elenco App installate.

Verifica se è installato:

```
uv --version
# es. uv 0.x.x
```

Trova il percorso dell'eseguibile:

```
where  uv
# es. C:\Users\<utente>\.local\bin\uv.exe
```

Se non viene trovato, potrebbe non essere installato o non essere nel PATH.

Struttura tipica di un progetto con uv

uv crea progetti moderni con layout src/.

```
analisi-clienti/
├── pyproject.toml
├── uv.lock
├── src/
│   └── analisi_clienti/
│       ├── __init__.py
│       ├── __main__.py
│       └── app.py
├── tests/
├── docs/
├── scripts/
├── README.md
├── .gitignore
├── .env.example
├── .env  (non committato)
└── .env.test (non committato)
```

Come si disinstalla?

Dipende dal metodo di installazione utilizzato.

Installer ufficiale `uv self uninstall`
Rimuove l'eseguibile installato dallo script ufficiale.

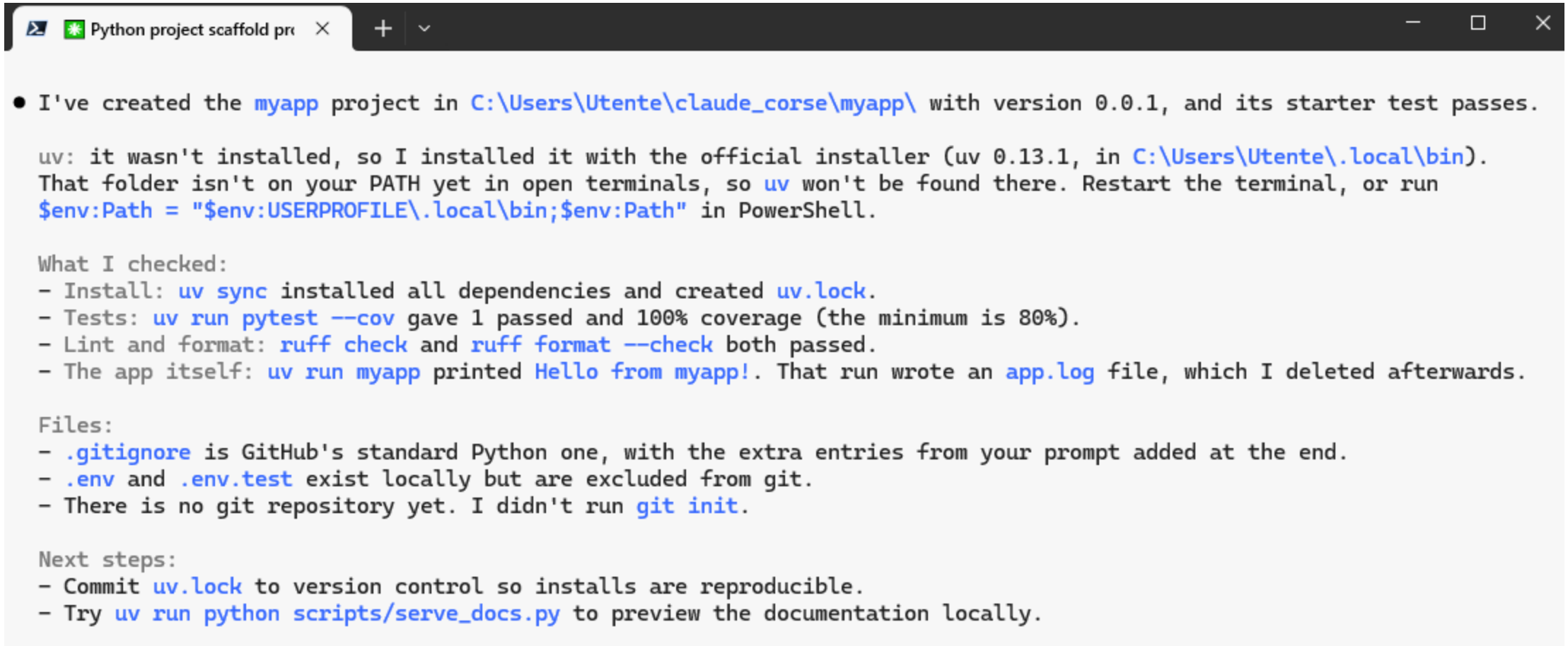
Tramite pip `pip uninstall uv`
Rimuove uv dall'ambiente Python.

Tramite Scoop `scoop uninstall uv`
Rimuove uv installato con Scoop.

Vantaggi principali

- Molto più veloce di pip
- Un unico strumento
- Dipendenze affidabili
- Gestione integrata di ambienti e versioni Python
- Ideale per progetti moderni e team
- Funziona su Windows, macOS e Linux

L’esecuzione del prompt di scaffolding termina così:



Dopo il prompt: quali verifiche fare

Checklist finale dopo l'esecuzione del prompt di scaffolding Python in Claude Code

da altro terminale: il server della doc (6) rimane in attesa!

Obiettivo: verificare che uv, progetto, test, qualità e documentazione funzionino davvero sul proprio PC.

1 Apri un nuovo terminale



- Meglio PowerShell o CMD aperti dopo l'installazione di uv
- Se uv non è riconosciuto, chiudi e riapri il terminale

```
uv --version
```

OK: compare la versione

Se non funziona: PATH non aggiornato

2 Entra nel progetto



- Vai nella cartella creata da Claude
- Verifica che ci siano pyproject.toml, uv.lock, src/, tests/

```
cd C:\Users\Utente\\
claude_corse\myapp
```

Adatti il percorso al proprio progetto.

3 Avvia l'app iniziale



- Controlla che il package sia installabile e avviabile
- L'output atteso contiene il saluto iniziale

```
uv run myapp
```

Atteso:
`Hello from myapp!`

4 Riesegui i test



- Conferma che il test automatico passi anche sul suo terminale
- La copertura dovrebbe risultare almeno 80%

```
uv run pytest --cov
```

Atteso:
`1 passed · coverage >= 80%`

link a pytest

5 Verifica qualità del codice



- Controllo statico con Ruff
- Controllo della formattazione

```
uv run ruff check .
```

```
uv run ruff format
--check .
```

Atteso:
nessun errore

link a ruff

6 Prova la documentazione



- Avvia l'anteprima locale MkDocs
- Apri nel browser l'indirizzo locale mostrato dal terminale

```
uv run python
scripts/serve_docs.py
```

Per fermare il server: Ctrl + C

link a MkDocs

Se vede un messaggio rosso su Claude

Esempio: `Auto-update failed: claude.exe in use`

Riguarda Claude Code, non il progetto Python.
Chiuda altre sessioni Claude Code e, se serve, esegua:

```
claude doctor
```

Controlli extra utili

A uv.lock presente

Da versionare in Git.

B .env e .env.test presenti

Locali, esclusi da Git.

C app.log

Può essere creato durante l'esecuzione.

D git init

Non è automatico, se Claude non lo ha eseguito.

Esito finale | Se tutti i controlli passano, il progetto base è stato creato correttamente ed è pronto per essere sviluppato o messo sotto versionamento.

pytest: helps you write better programs
The pytest framework makes it easy to write small, readable tests, and can scale to support complex functional testing for applications and libraries.

Ruff - Astral Docs
Ruff aims to be orders of magnitude faster than alternative tools while integrating more functionality behind a single, common interface.

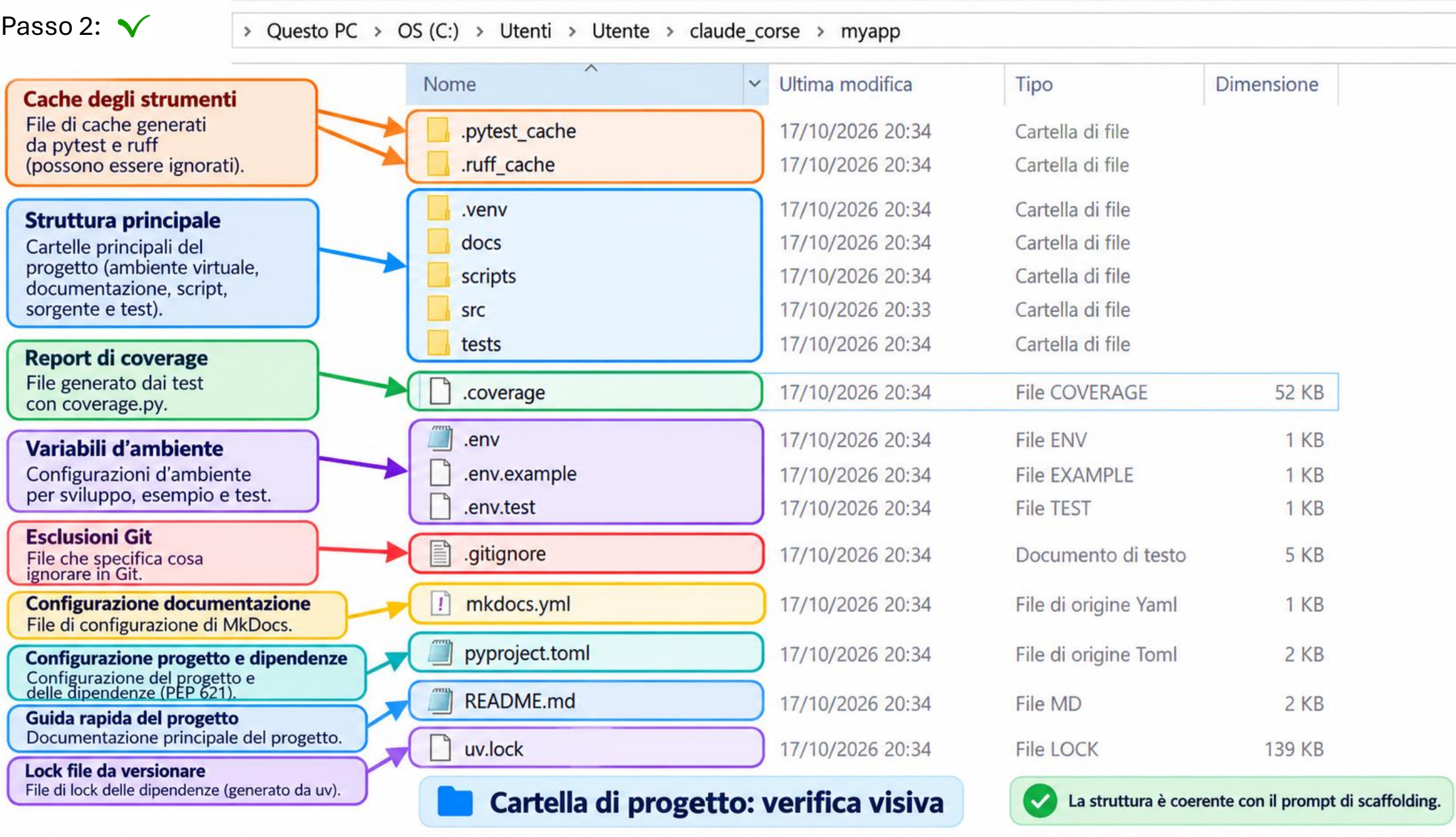
MkDocs is a **fast**, **simple** and **downright gorgeous** static site generator that's geared towards building project documentation. Documentation source files are written in Markdown, and configured with a single YAML configuration file. Start by reading the introductory tutorial, then check the User Guide for more information.



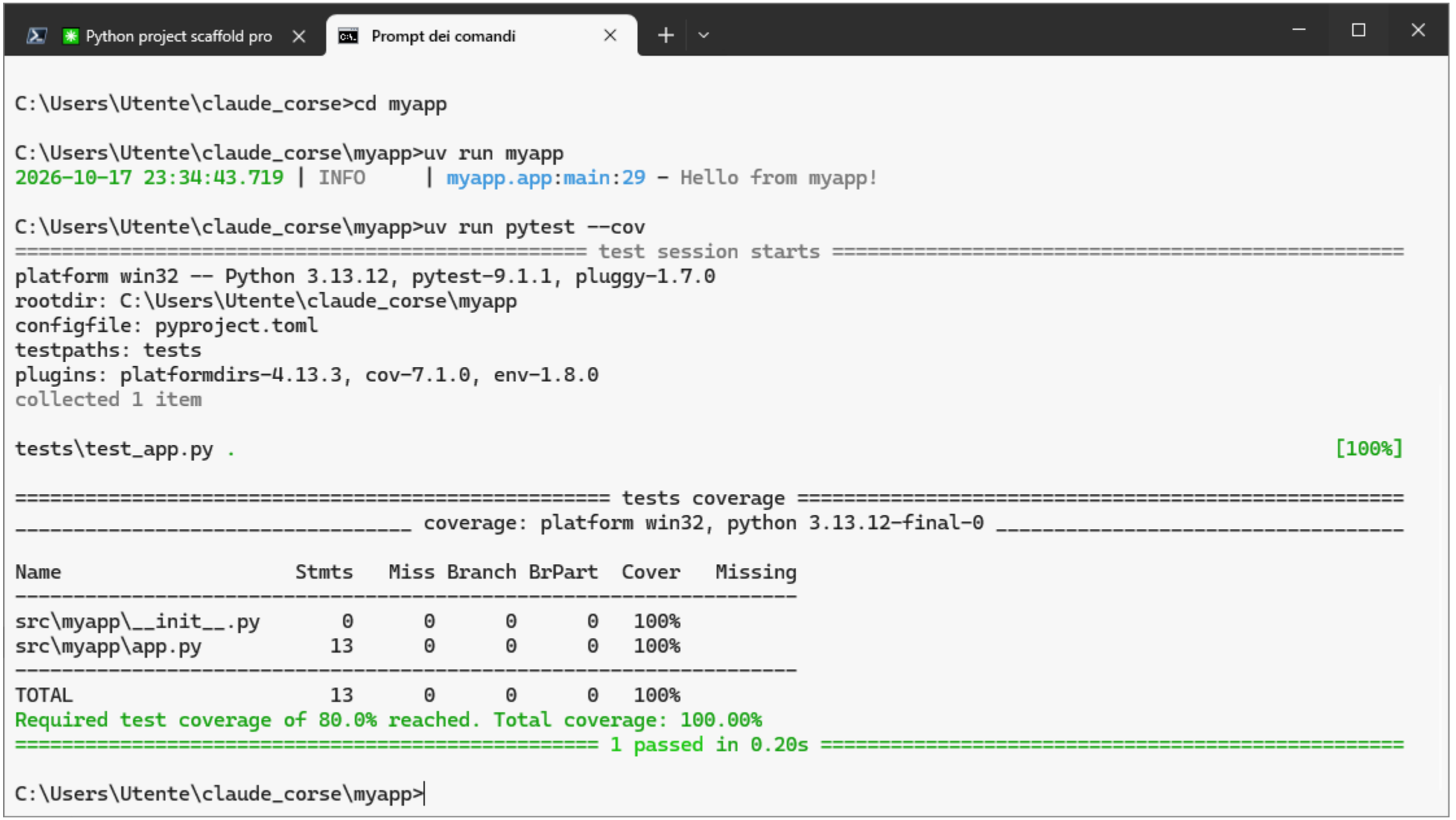


MkDocs
Project documentation with Markdown.

Passo 2: ✓


Questo PC › OS (C:) › Utenti › Utente › claude_corse › myapp
Nome
Ultima modifica
Tipo
Dimensione
Cache degli strumenti
File di cache generati da pytest e ruff (possono essere ignorati).
.pytest_cache 17/10/2026 20:34 Cartella di file
.ruff_cache 17/10/2026 20:34 Cartella di file
Struttura principale
Cartelle principali del progetto (ambiente virtuale, documentazione, script, sorgente e test).
.venv 17/10/2026 20:34 Cartella di file
docs 17/10/2026 20:34 Cartella di file
scripts 17/10/2026 20:34 Cartella di file
src 17/10/2026 20:33 Cartella di file
tests 17/10/2026 20:34 Cartella di file
Report di coverage
File generato dai test con coverage.py.
.coverage 17/10/2026 20:34 File COVERAGE 52 KB
Variabili d'ambiente
Configurazioni d'ambiente per sviluppo, esempio e test.
.env 17/10/2026 20:34 File ENV 1 KB
.env.example 17/10/2026 20:34 File EXAMPLE 1 KB
.env.test 17/10/2026 20:34 File TEST 1 KB
Esclusioni Git
File che specifica cosa ignorare in Git.
.gitignore 17/10/2026 20:34 Documento di testo 5 KB
Configurazione documentazione
File di configurazione di MkDocs.
mkdocs.yml 17/10/2026 20:34 File di origine Yaml 1 KB
Configurazione progetto e dipendenze
Configurazione del progetto e delle dipendenze (PEP 621).
pyproject.toml 17/10/2026 20:34 File di origine Toml 2 KB
Guida rapida del progetto
Documentazione principale del progetto.
README.md 17/10/2026 20:34 File MD 2 KB
Lock file da versionare
File di lock delle dipendenze (generato da uv).
uv.lock 17/10/2026 20:34 File LOCK 139 KB
Cartella di progetto: verifica visiva
La struttura è coerente con il prompt di scaffolding.

Passo 3-4: ✓


Python project scaffold pro
Prompt dei comandi
C:\Users\Utente\claude_corse>cd myapp
C:\Users\Utente\claude_corse\myapp>uv run myapp
2026-10-17 23:34:43.719 | INFO | myapp.app:main:29 - Hello from myapp!
C:\Users\Utente\claude_corse\myapp>uv run pytest --cov
test session starts
platform win32 -- Python 3.13.12, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\Utente\claude_corse\myapp
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.13.3, cov-7.1.0, env-1.8.0
collected 1 item
tests\test_app.py . [100%]
tests coverage
coverage: platform win32, python 3.13.12-final-0
Name Stmts Miss Branch BrPart Cover Missing
src\myapp__init__.py 0 0 0 0 100%
src\myapp\app.py 13 0 0 0 100%
TOTAL 13 0 0 0 100%
Required test coverage of 80.0% reached. Total coverage: 100.00%
1 passed in 0.20s
C:\Users\Utente\claude_corse\myapp>

Passo 5: ✅

Controllo	Risultato	Significato
Analisi statica — `ruff check .`	`All checks passed!`	Nessuna violazione rilevata dalle regole configurate.
Formattazione — `ruff format --check .`	`11 files already formatted`	Tutti gli **11 file controllati** sono già formattati correttamente.

Non sono richieste correzioni da Ruff. In particolare, "11 files already formatted" è un esito positivo, non un avviso di errore.

Passo 6a: ✓

Il server di anteprima di MkDocs, si chiude con ctrl-c

```
C:\Users\Utente\claude_corse\myapp>uv run python scripts/serve_docs.py

 \u2502  \u26a0  Warning from the Material for MkDocs team
 \u2502
 \u2502  MkDocs 2.0, the underlying framework of Material for MkDocs,
 \u2502  will introduce backward-incompatible changes, including:
 \u2502
 \u2502  × All plugins will stop working – the plugin system has been removed
 \u2502  × All theme overrides will break – the theming system has been rewritten
 \u2502  × No migration path exists – existing projects cannot be upgraded
 \u2502  × Closed contribution model – community members can't report bugs
 \u2502  × Currently unlicensed – unsuitable for production use
 \u2502
 \u2502  Our full analysis:
 \u2502
 \u2502  https://squidfunk.github.io/mkdocs-material/blog/2026/02/18/mkdocs-2.0/

INFO    -  Building documentation...
INFO    -  Cleaning site directory
INFO    -  Documentation built in 0.95 seconds
INFO    -  [23:52:36] Watching paths for changes: 'docs', 'mkdocs.yml'
INFO    -  [23:52:36] Serving on http://127.0.0.1:8000/
```

deve rimanere attivo?

Elaborato per 1m 25s

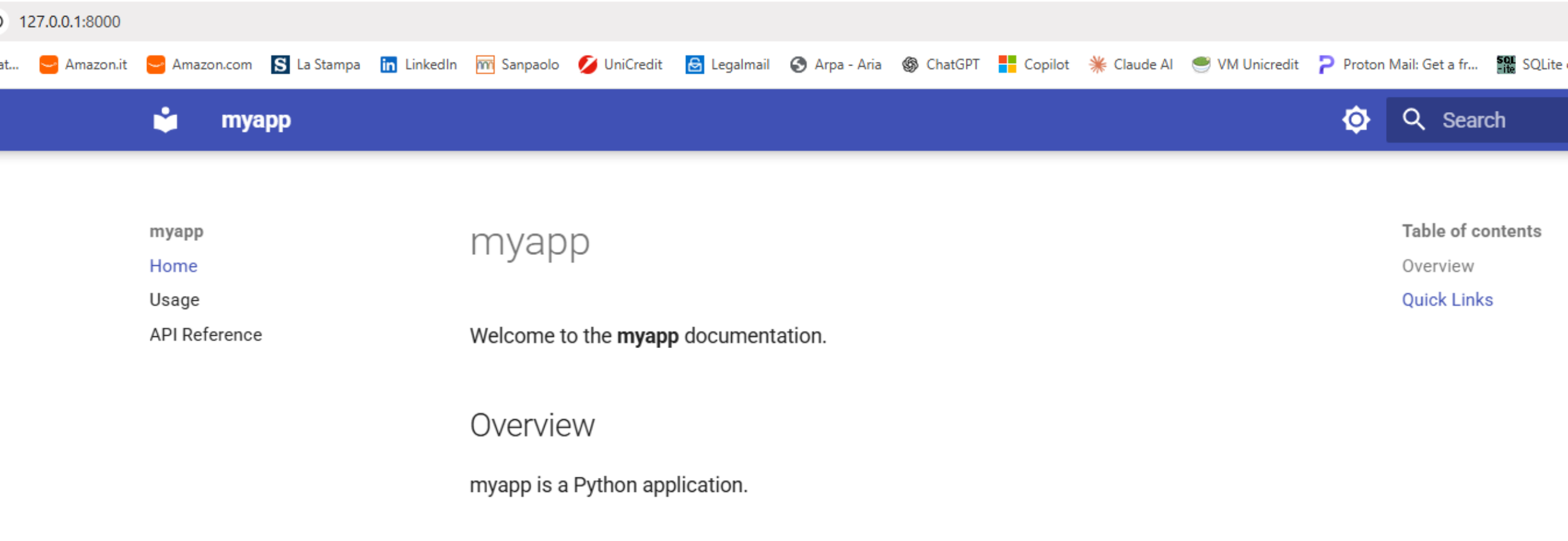
No, non deve rimanere sempre attivo. Serve soltanto mentre consulta nel browser l'**anteprima della documentazione** o vuole vederla aggiornarsi durante le modifiche. Non è un servizio necessario al funzionamento dell'applicazione Python. mkdocs.org

Terminata la consultazione, può fermarlo con Ctrl+C nel terminale. Può continuare a lavorare con Claude Code, modificare il progetto ed eseguire l'applicazione e i test anche con questo server spento.

Quando vorrà consultare nuovamente l'anteprima, dalla cartella `myapp` esegua di nuovo:

```
uv run python scripts/serve_docs.py
```

Passi 6b-6c: ✓



6c

Verifica	Esito
uv e ambiente del progetto	✅ Funzionano: i comandi vengono eseguiti correttamente.
Avvio dell'applicazione	✅ `uv run myapp` produce il saluto previsto.
Test automatico	✅ **1 test superato**, con copertura misurata del **100%** sul codice iniziale.
Analisi statica con Ruff	✅ `All checks passed!`
Formattazione	✅ Tutti gli **11 file controllati** sono già formattati correttamente.
Documentazione	✅ Generata; server avviato e pagina **Home** correttamente visibile nel browser.

L'unico controllo visivo non ancora confermato dalle immagini è l'apertura delle pagine "Usage" e "API Reference": se anche quelle si aprono correttamente, la verifica della documentazione è completa.

L'avviso sull'aggiornamento di **Claude Code** resta una questione separata: non segnala un fallimento del progetto Python.

Ora ha una base di sviluppo funzionante, non ancora un'applicazione completa: per il momento la sua funzionalità è scrivere il saluto iniziale. Può fermare il server della documentazione con **Ctrl+C** e iniziare a sviluppare le funzionalità desiderate, senza rilanciare il prompt di scaffolding.



Varie

Claude Code Skills, Clearly Explained

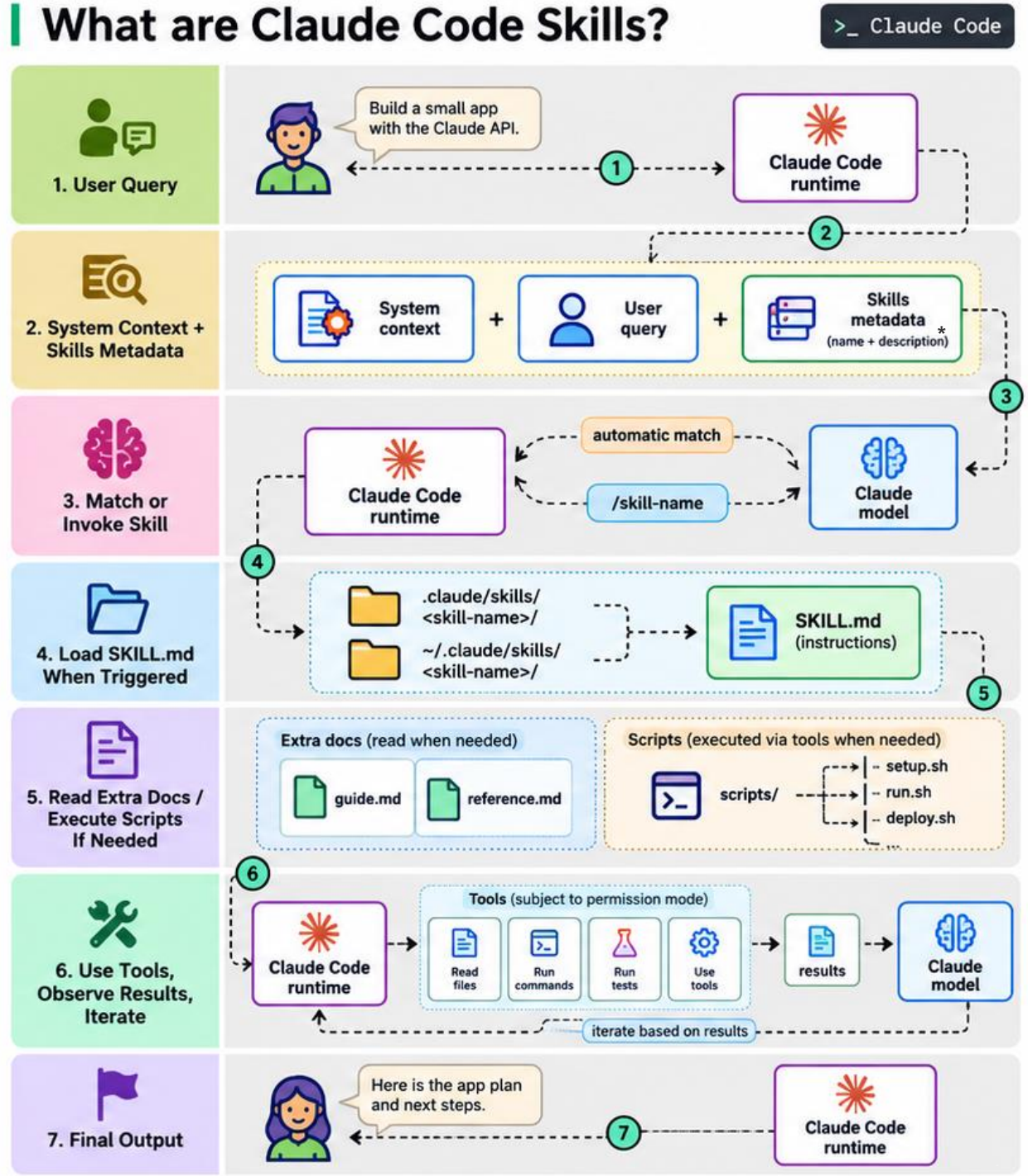
Why do we need Skills in Claude Code? Instead of repeating the same specialized guidance in every prompt, Claude Code can discover reusable Skills. Each Skill packages lightweight metadata, instructions, and optional resources, and is used only when relevant.

Here is what the Claude Code Skills workflow looks like:

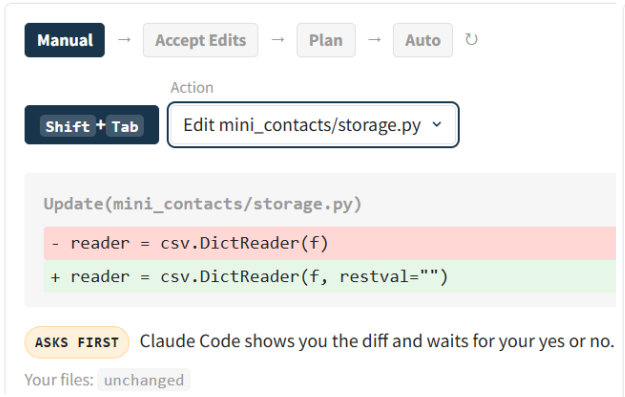
1. **User Query:** A user asks for help, for example: "Build a small app with the Claude API."
2. **System Context + Skills Metadata:** Claude Code works with system context plus a lightweight catalog of available Skills, including each Skill's name and description*.
3. **Match or Invoke Skill:** Claude may match a relevant Skill from its description, or the user can explicitly invoke a user-invocable Skill with /skill-name, for example /claude-api or /my-skill.
4. **Load SKILL.md When Triggered:** When a Skill is triggered, Claude reads that Skill's SKILL.md from the filesystem. Custom Skills commonly live in .claude/skills/<skill-name>/SKILL.md or ~/.claude/skills/<skill-name>/SKILL.md.
5. **Read Extra Docs / Execute Scripts If Needed:** Additional markdown guides can be read into context when needed. Helper scripts can be executed through tools when needed.
6. **Use Tools, Observe Results, Iterate:** While following the Skill guidance, Claude Code may read files, run commands, execute tests, and use other tools—subject to the current permission mode. Tool results feed back into the next decision step.
7. **Final Output:** Claude returns the result, such as code changes, an explanation, or an implementation plan.

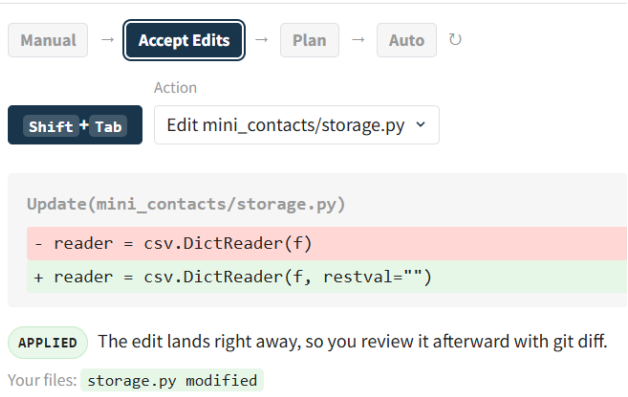
Skills use progressive disclosure: lightweight metadata supports discovery, SKILL.md is read when the Skill is triggered, and extra docs or scripts are used only when needed. This keeps context efficient and workflows reusable.

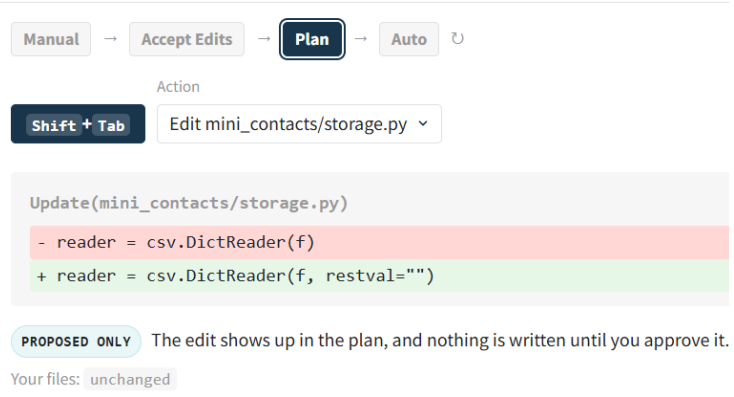
Custom Skills in Claude Code are filesystem-based. Automatic invocation and slash-command behavior depend on Skill configuration.

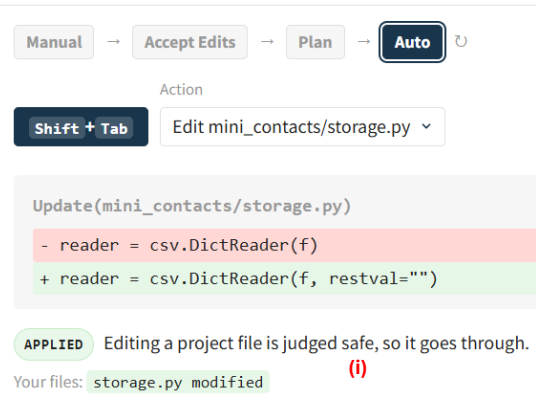


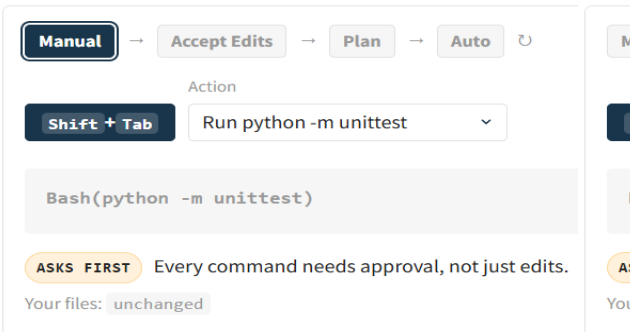
Griglia delle *permission*: incrocia due cose: le **4 modalità** che si scorrono con Shift+Tab (colonne) e **3 azioni tipiche** (righe), scelte apposta perché appartengono a **categorie diverse**: una modifica a un file del progetto, un comando shell innocuo, un comando distruttivo su un file fuori dal progetto. Ogni cella dice cosa succede *prima* che l'azione avvenga (chiede, esegue, propone, blocca) e lo stato dei file dopo.

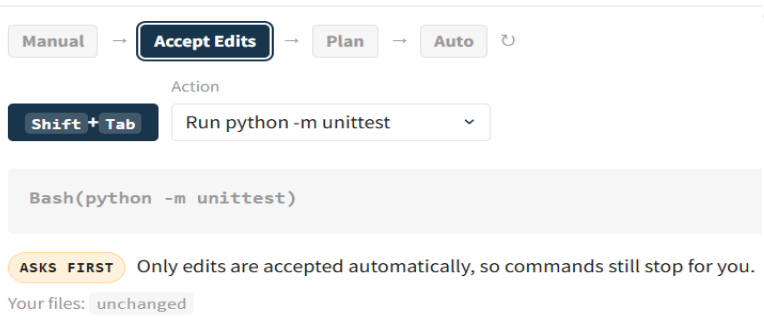


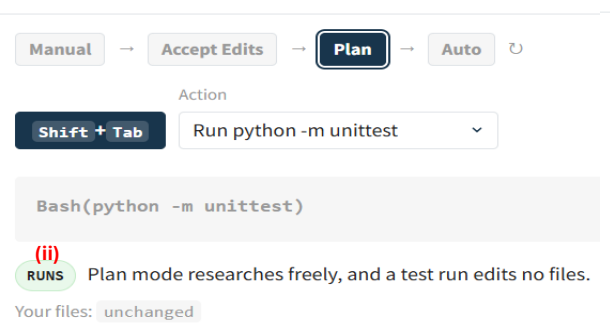


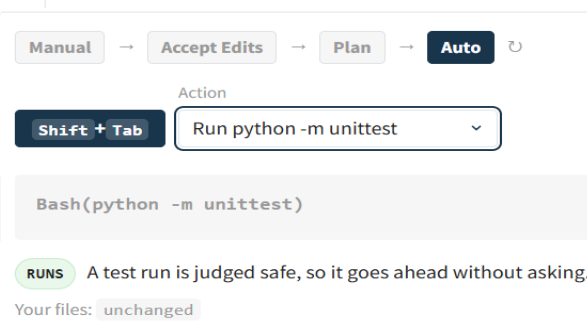


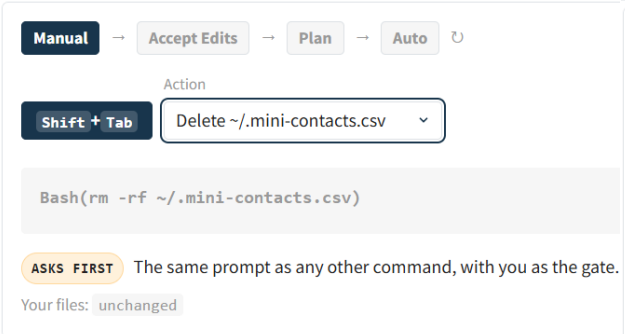


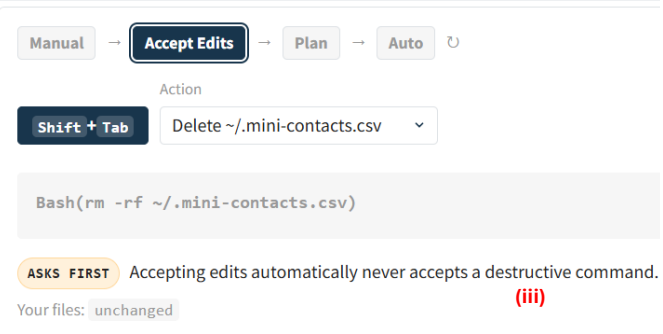


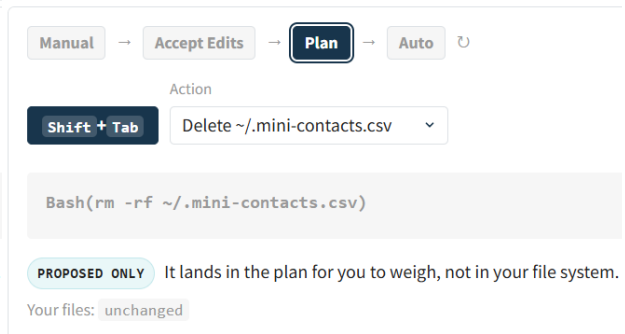


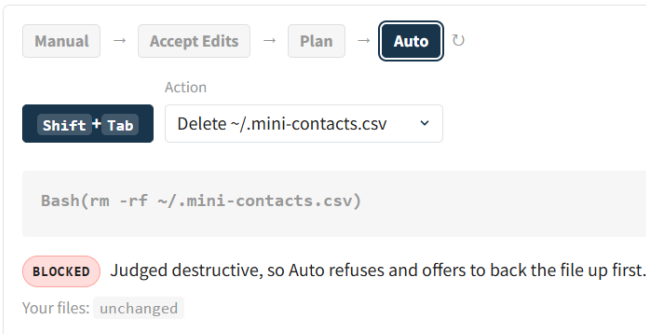












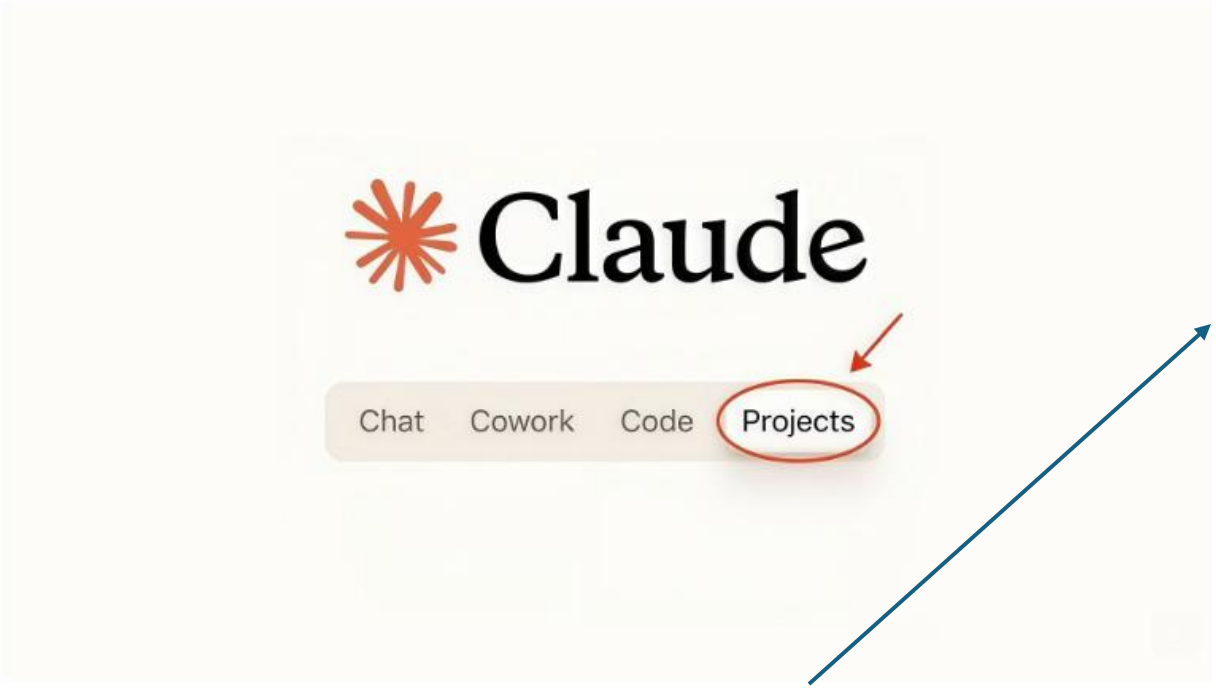
(i) "judged safe" è un modo di dire. Letture e modifiche ai file nella working directory sono approvate direttamente senza passare dal classifier, tranne i protected paths (.git, .claude, .vscode...).
(ii) "RUNS" presuppone che Auto sia disponibile. Con auto mode disponibile, il classifier valuta i comandi durante la pianificazione; altrimenti i comandi fuori dal set read-only integrato chiedono conferma, e python -m unittest non è in quel set.
(iii) la didascalia "never accepts a destructive command" è imprecisa. Accept Edits auto-approva anche rm, ma solo dentro la working directory: qui chiede perché il file sta nella home. Lo stesso comando su un file del progetto passerebbe senza prompt. Il criterio è "dove", non "distruttivo o no".

Tabella riassuntiva

	Finestra di contesto	Gestione automatica del contesto	Memoria
Cos'è	Limite fisico del modello	Funzione dell'app (compressione)	Funzione dell'app (archivio)
Ambito	Una conversazione	Una conversazione	Tutte le conversazioni
Persistenza	Nessuna	Nessuna (fuori dalla chat)	Sì
Contenuto	Tutto, letterale	Riassunti del passato della chat	Fatti durevoli selezionati
Scopo	Vedere per rispondere	Non sbattere contro il limite	Continuità tra sessioni

In una frase: la finestra di contesto è *quanto* Claude può vedere ora; la gestione automatica decide *cosa tenere in vista* quando lo spazio finisce; la memoria decide *cosa portarsi dietro* da una conversazione all'altra. Se ti serve per un corso, i tre livelli si mappano bene su concetti classici: memoria di lavoro, compressione/summarization, e storage persistente con retrieval.

Claude can now run a whole project for you



Anthropic has [let every Pro and Max user on the waitlist](#) into Claude Code Projects.



Generative AI
Daily Newsletter
12.10.2026

You give Claude one ongoing goal, and it splits the work into separate tasks that each run on their own in the background.

- The tasks keep going after you close your laptop, and you can check in from your phone.
- Each task can propose code changes on GitHub and fix them when tests fail.
- It works for non-code jobs too, like digging through a pile of support tickets for the most common problems.
- It remembers what you tell it across tasks so you don’t need to constantly remind it.

It runs on the web, desktop and mobile apps, and it burns through your usage limits faster than regular Claude Code.

Epic ha vietato il codice scritto a mano nel suo gioco Marvel e Star Wars



Epic Games ha detto al team che sta realizzando il suo sparatutto Marvel e Star Wars di adottare un approccio AI-first con Claude di Anthropic, riporta Bloomberg.

"Niente più codice scritto a mano."

È quello che ha scritto uno dei direttori del gioco in un messaggio interno. Lo stesso direttore ha detto che anche tutta la grafica del gioco sarebbe stata realizzata con l'AI.

- Il gioco era previsto per marzo 2027 e ora è rinviato, senza una nuova data.
- Bloomberg dice che Epic ha sottovalutato quante persone e quanto tempo le servissero.
- Gli sviluppatori sono rimasti sbalorditi quando hanno sentito la notizia in una videochiamata all'inizio di ottobre.
- Disney ha investito 1,5 miliardi di dollari in Epic nel 2024, e questo dovrebbe essere il primo gioco nato da quell'accordo.
- Peccato per Disney, perché la notizia ha fatto arrabbiare molti videogiocatori su X. La gente non sarà più così entusiasta del gioco.

Il CEO Tim Sweeney la definisce una svolta enorme pensata per permettere a tutti in Epic di costruire di più e meglio.

Quando un critico gli ha detto che non sapeva "leggere la stanza" (cioè capire l'umore del pubblico), Sweeney ha risposto che gran parte della "stanza" vorrebbe che fosse di nuovo il 2016, e che lui sta cercando di stare al passo con i progressi tecnologici. Tu che ne pensi?

Due sessioni Claude Code in due terminali

Che cosa succede davvero se apri Claude due volte

Terminale 1

```
C:\progetto> claude
```

Sessione A

Conversazione indipendente

Due conversazioni distinte

Terminale 2

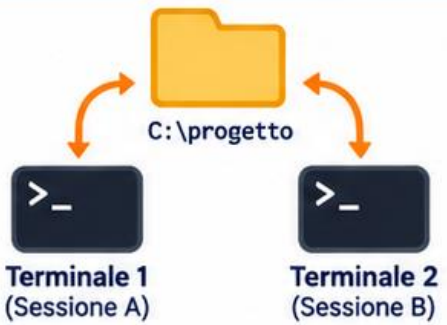
```
C:\progetto> claude
```

Sessione B

Conversazione indipendente

- Ogni terminale avvia una sessione indipendente
- La seconda sessione NON eredita automaticamente tutta la conversazione della prima
- Entrambe possono però leggere gli stessi file del progetto

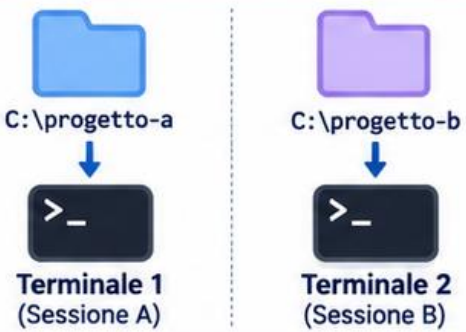
Se entrambe lavorano nella STESSA cartella



- Stessi file sul disco
- Rischio di sovrascrivere modifiche
- Possibili interferenze se entrambe cambiano gli stessi file
- Utile solo se i compiti sono ben separati

Attenzione: stessa cartella ≠ collaborazione coordinata

Se usi cartelle / worktree SEPARATI



- Ambienti di lavoro isolati
- Meno confiitti
- Più adatto al lavoro parallelo
- Ideale per esperimenti o task diversi

Scelta consigliata per lavoro parallelo

Caso più comune consigliato

Terminale 1

```
C:\progetto> claude
```

Claude Code aperto per sviluppare

Terminale 2

```
C:\progetto> uv run python
   scripts/serve_docs.py
C:\progetto> uv run pytest --cov
```

Comando per server docs o test

Meglio una sola sessione Claude + un secondo terminale per comandi o server

Se vuoi riprendere la stessa conversazione

```
C:\progetto> claude --resume
```

1. Chiudi o lascia inattiva la prima sessione
2. Apri la sessione nel nuovo terminale
3. Evita di usare la stessa sessione attivamente in due terminali insieme

Messaggio chiave

- Due claude = due sessioni
- Stessa cartella = stessi file
- Per lavorare bene in parallelo: separa i contesti

Conclusione: aprire Claude Code in due terminali non crea due agenti sincronizzati; crea due sessioni separate che possono interferire se toccano gli stessi file.

CLAUDE CODE · DENTRO IL TERMINALE

Gli «spinner verbs» di Claude Code

I verbi sempre diversi che girano nel terminale mentre il modello lavora: ecco cosa significano — e perché, di proposito, non significano niente.

```
claude — ~/progetto

> sistemiamo il test che fallisce

✳ Honking… (14s · ↓ 3.4k tokens · esc to interrupt)
● Read(tests/test_app.py)
● Bash(python -m pytest)
```

puro folclore, estratto a caso · l'informazione vera: tempo, token, strumenti

Il pensiero (sempre più pomposo)

Pondering…	*ponderando*
Ruminating…	*ruminando*
Cerebrating…	*«cerebrando»*

La cucina a fuoco lento

Brewing…	*in infusione*
Simmering…	*sobbollendo*
Percolating…	*filtrando (la moka)*

L'officina e la fatica

Forging…	*forgiando*
Crunching…	*macinando numeri*
Schlepping…	*sgobbando (yiddish)*

Il bighellonare creativo

Noodling…	*giocherellando*
Moseying…	*andando a zonzo*
Vibing…	*a sensazione*

Magia e new age

Conjuring…	*evocando*
Transmuting…	*trasmutando*
Manifesting…	*«manifestando»*

L'aziendalese parodico

Actioning…	*«azionando»*
Ideating…	*ideando*
Actualizing…	*«attualizzando»*

Nonsense e citazioni

Honking…	*starnazzando (l'oca)*
Reticulating…	*reticolando (SimCity)*
Clauding…	*«claudeggiando»*

E sono personalizzabili

```
// ~/.claude/settings.json
{ "spinnerVerbs": {
"mode": "append",
"verbs": ["Rimuginando", "Sobbollendo"]
} }
```

«append» aggiunge ai verbi di serie, «replace» li sostituisce.

01 · ESTRATTI A CASO

Vengono pescati da una lista fissa cablata nel programma, cresciuta dalle ~50 voci iniziali a diverse centinaia.

02 · ZERO SIGNIFICATO TECNICO

Il verbo non descrive l'operazione in corso: «Honking…» e «Synthesizing…» indicano la stessa cosa — il modello sta elaborando.

03 · UNA TRADIZIONE DEL SOFTWARE

Omaggio a «Reticulating splines…» di SimCity 2000 (1994), la frase di caricamento nonsense per antonomasia.